The Journal of Supercomputing (2025) 81:1302
https://doi.org/10.1007/s11227-025-07723-8

RIMS-RPL: RL-based intelligent mobility-support of RPL for IIoT-based networks

Ismahene Alem[1] · Samira Yessad[1] · Louiza Bouallouche-Medjkoune[1]

Accepted: 30 July 2025

Abstract
Mobility is crucial in the industrial internet of things (IIoT), while the widely used IPv6 routing protocol for low-power and lossy networks (RPL) faces performance issues in the presence of mobile nodes (MNs). This paper presents methods to enhance mobility support and reliability in IIoT-based mobile networks while minimizing network overhead, energy consumption, and latency. Using a new cross-layer metric, sensor nodes select their next hop based on an objective function (OF) that considers node mobility. Additionally, a reinforcement learning (RL) approach allows nodes to predict their operational state -whether they are undergoing hand-offs, retaining outdated routes, or in a stable state- facilitating timely updates and reducing failures. Our protocol demonstrates lower computational complexity and superior performance compared to RPL and existing approaches under various mobility speeds and node densities. These results highlight the effectiveness and lightweight design of our proposed protocol for IIoT applications.

Keywords RPL · IIoT · Lightweight protocol · Reinforcement Learning · Mobility management · Objective Function

✉ Ismahene Alem
ismahene.alem@univ-bejaia.dz

Samira Yessad
samira.yessad@univ-bejaia.dz

Louiza Bouallouche-Medjkoune
louiza.medjkoune@univ-bejaia.dz

[1] Université de Bejaia, Unité de Recherche LaMOS, Faculté des Sciences Exactes, 06000 Bejaia, Algeria

Published online: 02 September 2025

1 Introduction

The integration of the internet of things (IoT) and artificial intelligence (AI) into manufacturing systems has considerably improved automation in industrial settings, leading to the development of what we now recognize as an IIoT and Industry 4.0-based smart factory system. IIoT is a subset of the broader IoT and is primarily intended for industrial applications. It encompasses connected items such as machines, collaborative robots (cobots), engines, and even products, all equipped with various types of sensors, including flow sensors, force sensors, position sensors, pressure sensors, temperature sensors, etc. [1]. Industrial wireless sensor networks (IWSNs) play a fundamental role as the backbone in IIoT-based systems [2]. The resulting systems efficiently monitor industrial environments and collect, exchange, analyze, and store information, without or with minimal human intervention, allowing better flexibility and efficiency in manufacturing processes [3].

In IIoT, communication between industrial devices enables the exchange of information, facilitating remote supervision and decision-making to improve production operations. However, the ability to implement IIoT technologies on a large scale is limited by small batteries, low memory, and limited processing capacity of sensors [4–6]. These devices, part of networks classified as low power and lossy networks (LLNs), face key challenges in communication aspects such as latency, throughput, reliability, and energy efficiency [7]. In this context, the internet engineering task force (IETF) working group has designed RPL to satisfy the long-term requirements for urban [8], industrial [9], home automation [10], and building automation applications [11].

In IIoT-based systems, MNs play a pivotal role in various automation scenarios. They can be attached to products, workers, robots, and vehicles to provide essential industrial services such as real-time tracking, enhancing visibility and efficiency in logistics, supply chain operations, and assembly processes (see Fig. 1). However, RPL was designed for static IoT-based networks and suffers from various mobility weaknesses because the dynamic nature of MNs leads to outdated routing information and frequent changes in network topology along with link failures. This has motivated researchers to develop adaptive approaches that aim to ensure efficient communication between stationary and mobile IIoT devices. Recent advancements have integrated machine learning (ML) techniques to enhance RPL's adaptability in dynamic environments (see, for example, [12–15]), along with software-defined networking (SDN)-based reconfigurable edge architectures that improve IIoT flexibility and latency under dynamic conditions [16]. Among ML-based approaches, RL-based protocols have shown their effectiveness in various industrial domains, such as control systems, robotics, energy management, and autonomous vehicles [15–18]. RL, a psychology-inspired technique based on historical experiences, enables an agent (or decision-maker) to efficiently learn to make decisions through interaction with its environment, solving complex optimization problems [19, 20].

In this paper, we propose RL-based intelligent mobility-support for RPL in IWSNs (RIMS-RPL), an effective and mobility-aware routing protocol designed

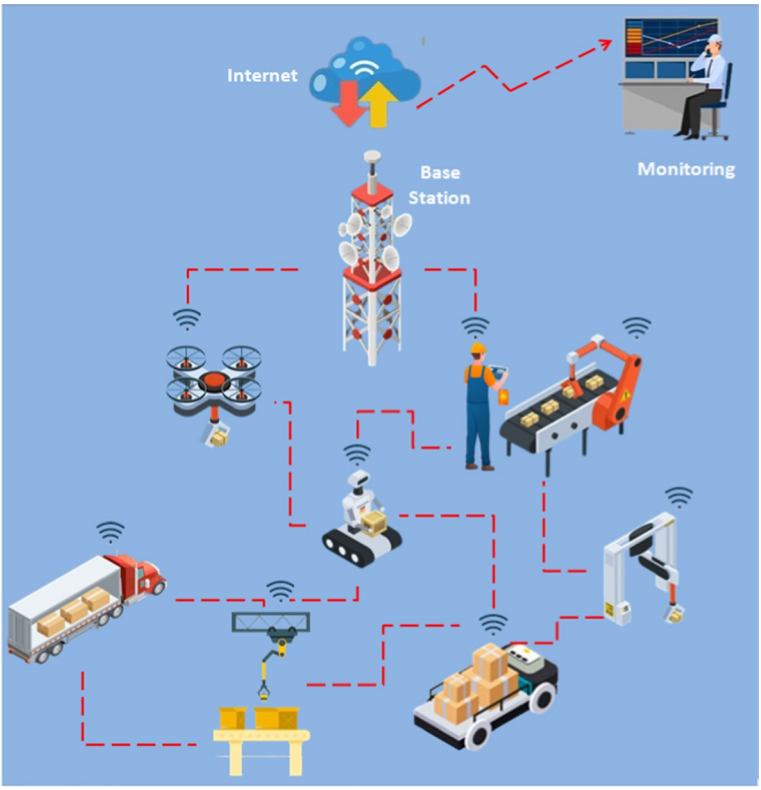

Fig. 1 Example of an IIoT-based mobile network

to accommodate node mobility in IIoT-based networks. The aim of RIMS-RPL is to maintain continuous connectivity for sensor nodes by predicting their mobility, refreshing routing information before link failures occur, and ensuring reliable communication paths while minimizing packet delivery delays and energy consumption. The major contributions of this work are listed below.

1. Introduction of a new cross-layer metric, that is, the expected reliability percentage (ERP), to detect mobility and enhance reliability in IWSNs. This involves the exchange of information between the routing layer (RPL) and the medium access control (MAC) layer (Carrier sense multiple access with collision avoidance (CSMA/CA)), allowing the monitoring of both successfully transmitted packets and those that failed.
2. Proposition of a new parent selection procedure that integrates the ERP metric with other critical metrics to improve the reliability and robustness of routing paths.
3. Provision of an RL-based algorithm utilizing the proposed metric and the frequency of parent changes to predict the current node's state, i.e., whether it is

undergoing hand-off processes (too many parent changes in a short period), retaining outdated routing information, or in a stable situation. Based on this prediction, the agent makes decisions such as sending a destination oriented directed acyclic graph (DODAG) information solicitation (DIS) message to promptly refresh its routing information, selecting a new parent after removing the old one and resetting the trickle timer to update the parent list, or continuing processing without taking any action, respectively.
4. Complexity calculation of parent selection and RL-based algorithms.
5. Real-world use case to demonstrate the practical applicability of the proposed protocol, addressing potential implementation challenges.
6. Comparison of RIMS-RPL against the baseline RPL and reference benchmark protocols, through a set of experiments in an IIoT-based network via the Cooja simulator. The results demonstrate that our protocol outperforms the state-of-the-art in terms of packet delivery ratio (PDR), latency, control overhead, and energy consumption.

The remainder of the paper is organized as follows: Sect. 2 provides a background on the standard RPL and outlines its limitations regarding mobility. Next, Sect. 3 presents a brief overview of related works. Following that, Sect. 4 introduces the problem formulation and elaborates on the motivation behind our approach. Subsequently, Sect. 5 details the proposed RIMS-RPL, including a complexity analysis of its underlying algorithms. In Sect. 6, we describe the system setup and present a comprehensive performance evaluation. Then, Sect. refsec:RealspsWorldUseCaseScenario discusses a real-world use case, including potential implementation challenges. Finally, Sect. 8 concludes the paper and outlines directions for future research.

2 Background on RPL

RPL is a hierarchical IPv6 routing protocol designed for LLNs. It was introduced by the IETF working group and standardized in March 2012 as RFC 6550 [21]. The RPL routing protocol supports various traffic patterns, including point-to-point (P2P), point-to-multi-point (P2MP), and multi-point-to-point (MP2P), and is used in diverse domains such as industrial, urban, and home automation [22–24]. It is designed for resource-constrained devices with limitations, such as low memory, minimal data traffic, low-power radios, and limited battery life. RPL aims to establish a tree-shaped topology with a single root (or sink) known as DODAG, which connects IoT nodes to the root to deliver their data over multi-hops. To determine optimized routing paths and establish connections between IoT nodes, RPL employs an OF within each node. This function incorporates a set of node/link metrics and constraints, allowing the selection of the best parent (BP) for each node. The OF is designed to meet the specific quality of service (QoS) requirements of each IoT application, such as energy efficiency, low latency, and efficient data delivery.

In this regard, four types of control messages should be exchanged between nodes to construct the directed acyclic graph (DAG), which are defined as follows:

- DODAG Information Object (DIO): advertises OF information;
- DODAG Information Solicitation (DIS): solicits the sending of DIO and notifies existing nodes of new nodes' intent to join the DODAG;
- Destination Advertisement Object (DAO): propagates routing destination information upward in the DODAG, facilitating downward traffic in P2MP and P2P traffic patterns. Each node sends a DAO message to its BP to establish and maintain reverse paths, ensuring effective routing within the network.
- Destination Advertisement Object Acknowledgment (DAO-ACK): employed by the receiver to acknowledge the reception of the DAO message.

Among the control messages previously listed, the DIO message is paramount in discovering appropriate upward routes. Initiated by the DODAG root, the DIO message is periodically transmitted to broadcast OF information. This information enables nodes to transform metric values into ranks, providing an approximation of the node's distance from the sink, which holds the lowest rank, and preventing loops within the RPL structure. Typically, the BP, also known as the preferred parent (PP), is identified as the one with the lowest rank compared to its candidate parents (CPs). Figure 2 shows an example of the RPL routing DODAG and its control messages.

To enhance the efficiency of the RPL control message exchanges, the trickle timer mechanism is used to schedule the transmission intervals of DIO messages. It ensures efficient communication and maintenance of the RPL-based network by dynamically regulating the frequency of DIO transmissions based on network stability, thereby reducing control overhead and conserving network resources.

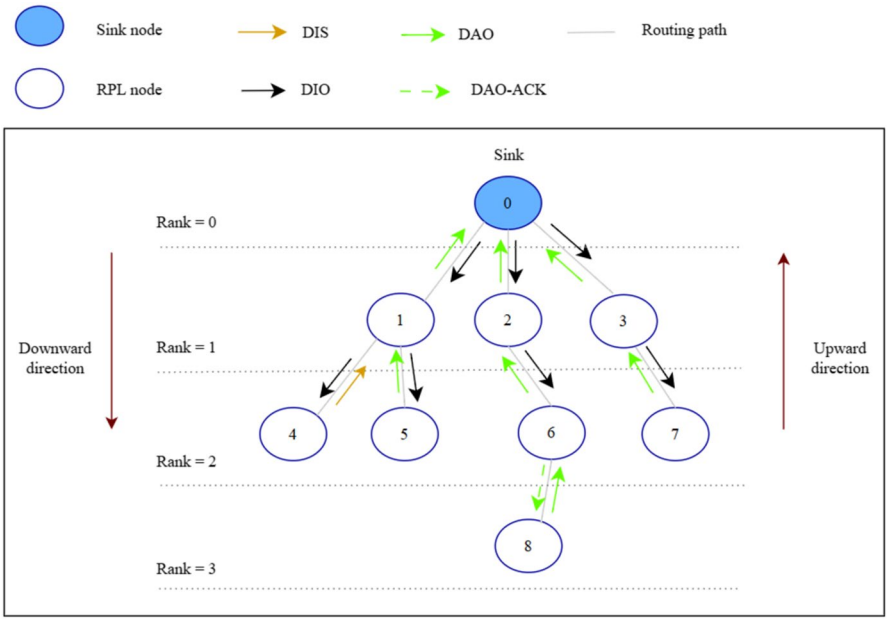

Fig. 2 Example of RPL routing DODAG and the transmission of control messages

Specifically, the time interval between two consecutive DIO messages increases exponentially as long as the RPL network remains stable, that is, when there are no changes, such as parent switches, modifications to the parent set, or rank adjustments. However, upon detecting any inconsistency or change, a node immediately resets its Trickle Timer to the minimum interval, thereby increasing the frequency of its DIO transmissions to quickly propagate updated network information.

RPL was originally designed for static networks and does not address routing management in mobile environments. A key challenge with the trickle timer in such environments arises when there are excessive parent changes. In an effort to improve resilience to mobility, the trickle timer increases the rate of control packets (shorter intervals). This heightened frequency results in increased communication and energy overhead, ultimately reducing the network's lifetime. Conversely, when the same parent remains unchanged for an extended period, longer intervals may cause nodes to miss important control messages needed to adjust routes due to mobility. Thus, the trickle timer in RPL is not well-suited for mobility in IIoT-based networks.

In response, we propose a cross-layer metric, a new parent selection procedure, and an RL-based approach for intelligent mobility management. This adaptation is designed to facilitate the mobility of the nodes, improve reliability, and reduce network overhead and energy consumption in critical applications within industrial scenarios.

3 Related works on addressing mobility in RPL

The literature contains numerous research efforts aimed at improving mobility support within RPL. In this section, we explore some of these research works, highlighting their advantages and disadvantages.

In [25], Sanshi and Jaidhar proposed an enhanced routing protocol for LLNs (ERPL) to support mobility. The core of ERPL involved designing a new OF while considering various routing metrics such as the expected transmission count (ETX), residual energy, and distance metrics. When selecting the PP node for the MNs, the direction of the MN's movement is also taken into account to quickly update the MN's PP when the node moves away from the previous parent. ERPL prevents MNs from becoming PP nodes to reduce retransmissions and control packet overhead. However, this strategy requires that MNs must be within the range of at least one fixed routing node (FRN), which could potentially limit the network's scalability and coverage area, especially in scenarios with sparse FRN deployment. Moreover, this approach may result in increased congestion on FRNs, leading to higher packet loss.

In [26], Lalani et al. proposed a mobility-aware objective function, called REFER, for IPv6 RPL. They addressed the issue of outdated parents in the limited neighbor table of MNs to facilitate successful handover between the previous and the new parent nodes. REFER introduced a new neighbor placement mechanism based on the parent's leasing time. CPs were selected based on metrics such as Euclidean distance, ETX, expected life time (ELT), and received signal strength index (RSSI), with priorities assigned accordingly. Additionally, the REFER protocol utilizes

the movement status of the nodes to prioritize fixed nodes as PP over MN, thus improving route stability. The simulation results demonstrate that REFER reduces energy consumption while simultaneously improving the network reliability. However, employing a large set of metrics requires more memory for management and increases the size of control messages required to transport these metrics. In addition, REFER compares CPs based only on their individual metric values rather than the total accumulated path cost. This localized decision-making can result in the selection of suboptimal routes, which contributes to increased latency.

Murali and Jamalipour [27] proposed a new parent selection algorithm aimed at supporting random mobility and improving energy efficiency in RPL. This algorithm uses the same set of metrics as in [26] to enhance parent node selection. In addition, they introduced an improved version of the trickle algorithm, called D-trickle, to address the issue of long listen-only periods in the basic RPL trickle algorithm. D-trickle dynamically allocates timers for sending DIO messages based on the number of neighboring nodes, thereby reducing network overhead. The simulation results show that the proposed protocol outperforms a range of existing RPL-based algorithms in terms of PDR, number of living nodes, and energy efficiency. However, it is important to note that the evaluation did not account for network overhead.

In [28], the authors introduced a novel mechanism to enhance RPL performance under mobility conditions. In particular, they focused on maintaining uninterrupted connectivity for MNs and optimizing parent selection using metrics such as ETX, RSSI, and residual energy (RE). The PPN plays a key role in this process, actively detecting the MN by periodically assessing RSSI variations after receiving a set number of data packets from the MN and then informing the MN to conserve energy and minimize network overhead. Once the MN becomes aware of its mobility, it broadcasts DIS messages to receive DIO messages from its neighbors and subsequently updates its routing information. Furthermore, the proposal incorporates a memory mechanism for MNs that stores previous parent selections, prioritizing stable parents. The simulation results demonstrate that the proposed strategy reduces the frequency of the parent switches and ensures reliable connectivity. Nevertheless, relying solely on RSSI for mobility detection may not be reliable in real-world scenarios due to potential interference.

In [29], the authors suggested adding a mobility timer (MT) to the standard RPL to facilitate seamless connectivity during mobility. The MT represents the expected duration for an MN to remain within radio range of its CP node. It is determined by considering the maximum achievable speed of the MN and the distance between the MN and its CP, which is derived from the RSSI value. When the MT of the current PP expires, a new PP is chosen from the CP's list using a fuzzy inference system. This system incorporates various metrics such as ETX, RE, RSSI, and MT. As in [25], MNs are prevented from becoming PPNs by restricting them from sending DIO messages, which also leads to the same problem. Furthermore, the accuracy of the MT is compromised as it relies on RSSI, which can be influenced by obstacles and interference.

In [30], the mobility enhanced RPL (ME-RPL) protocol introduces novel mechanisms to support mobility, addressing various challenges encountered by RPL. These challenges include the absence of MN identification, link failures

resulting from MNs being chosen as PPs, and packet loss caused by MNs detaching from the DODAG. Similarly to REFER in [26], ME-RPL incorporates mobility status into DIO messages, assigning priorities when selecting PP; for example, FRNs have a higher priority as PPs than MNs to improve reliability. Furthermore, node mobility is detected by monitoring changes in PPs. In cases where there is an increase in PP changes, the MN accelerates the transmission of DIS messages, facilitating nodes to update their routing information through DIO messages. Conversely, if the PP remains unchanged for several inter-DIS periods, the speed of DIS messages is reduced. However, in IoT-based networks with high mobility, a static PP for MNs over several intervals may indicate outdated information, and sending DIS messages may not resolve the issue (see Sect. 4). Moreover, an increase in PP changes prompts the MN to dispatch a higher volume of DIS messages in a short period. This action subsequently leads to an increase in DIO messages, ultimately leading to network overhead and congestion.

Seyfollahi et al., in [31], proposed a novel mechanism to improve reliability in healthcare applications based on the internet of medical things (IoMT). Principally, a loop prevention mechanism is introduced, allowing MNs to function as both routers and parent nodes within the network. Additionally, a dynamic adjustment mechanism is employed to modify the protocol's behavior when MNs remain stationary for regular intervals, thereby reducing control overhead and improving energy efficiency. The approach also improves parent selection by incorporating a new OF that uses the average RSSI of the five most recent packets received from each neighbor as a routing metric. However, relying solely on the average RSSI value for path selection can lead to suboptimal parent choices, particularly in dynamic or noisy wireless environments. Furthermore, control overhead is not explicitly evaluated, which is a key performance factor in resource-constrained IoT deployments.

In [32], the mobility aware RPL (MARPL) protocol introduces mobility support for the RPL protocol by using a novel metric called neighbor variability, which is used to detect node mobility, identify route disconnections, and adjust DIO and DIS message transmissions. The simulation results demonstrate improved performance compared to several benchmark protocols. However, MARPL relies on frequent RSSI-based monitoring triggered by the sensor's data packet period (DPP), which introduces limitations similar to those reported in [28, 29, 31]. Specifically, in environments with obstacles or interference, RSSI variations may not reliably indicate mobility, potentially resulting in inaccurate detection. Furthermore, MARPL assumes a uniform DPP across all nodes, limiting its applicability in heterogeneous networks where sensor nodes transmit data at different rates.

Although these presented studies have proposed improvements to the RPL protocol, we noticed that these methods have significant drawbacks:

- The majority of these protocols fail to capture real-time mobility variations since they use periodic updates, which result in outdated routing information.
- Their reliance on RSSI values for mobility detection often leads to inaccurate or unreliable results due to environmental obstacles and signal interference.

- The significant computational demands, latency, and control message overhead associated with mobility detection mechanisms limit their applicability to IIoT nodes, which typically have limited processing power and energy resources.

Our approach, RIMS-RPL, fills these gaps by integrating an adaptive mobility support mechanism based on a novel cross-layer packet loss metric ERP, anticipates mobility problems before they impact performance using a proactive RL system, and guarantees a lightweight and energy-efficient design for suitable applicability in IIoT settings.

4 Problem formulation and motivation

The integration of MNs is crucial for many industrial IoT-based applications. However, the standard RPL format proposed by the IETF working group does not account for mobility in its design. Specifically, when an MN moves away from its parent, two scenarios can arise. The first is the phenomenon of outdated routing information, while the second is the phenomenon of hand-off processes. These two scenarios have been simulated and illustrated in Figs. 3 and 4.

In Fig. 3, the network is organized according to a DODAG topology. The network consists of several nodes organized by rank. Node 0 (ID 0) serves as the fixed sink (DODAG root) with a rank of 0. Nodes 1 and 3 are fixed nodes with rank 1. Nodes 4, 5, 6, and 7 are also fixed and assigned rank 2, while node 8 is fixed with rank 3. Node 2 (ID 2) is an MN that initially (time $= T_0$) maintains rank 1 and is connected to the sink (Node 0) as its PP ($PP = 0$). At time T_1, the MN (Node 2) begins to move downward in the topology, away from the sink, toward a new physical location, which it reaches at T_2. During this movement, the MN is unable to

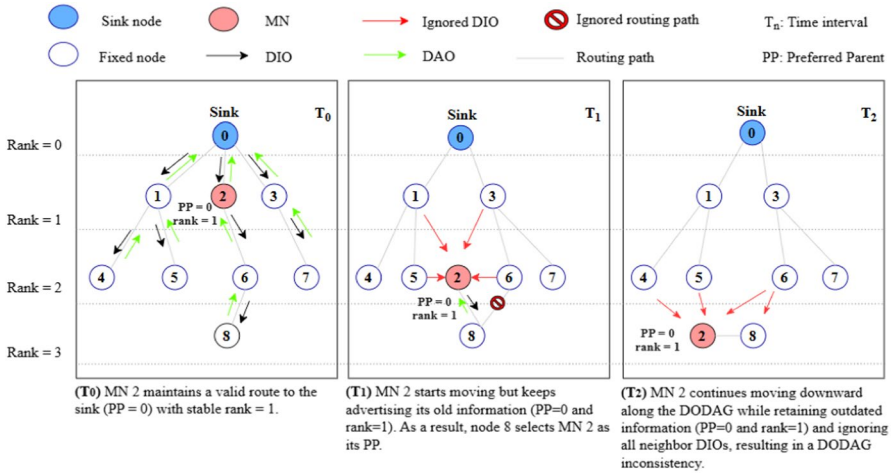

Fig. 3 Scenario illustrating the phenomenon of outdated routing information involving MN 2 mobility across time steps T_0, T_1, and T_2

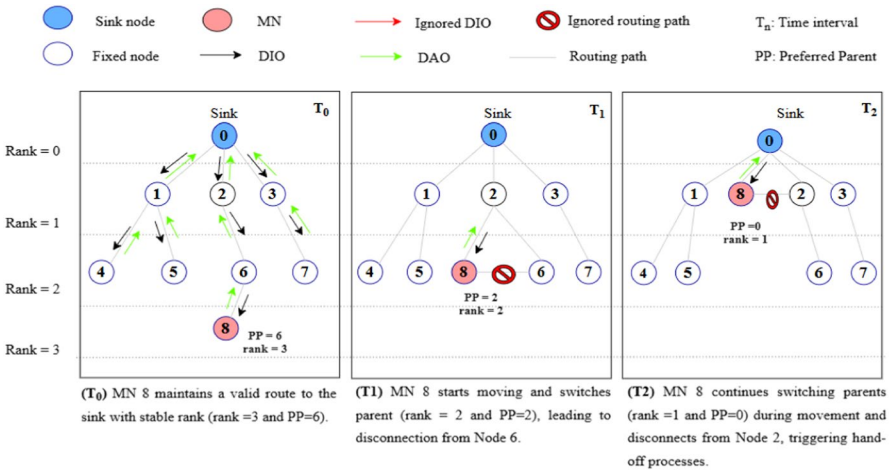

Fig. 4 Scenario illustrating the hand-off phenomenon involving MN 8 at time steps T_0, T_1, and T_2

immediately detect its mobility and continues to maintain its connection to its previous PP ($PP = 0$), ignoring any DIO messages received from the new neighbors. This delay occurs because the MN's current rank (1) remains lower than or equal to those of its new neighbors, preventing timely parent reassessment. Meanwhile, the fixed nodes in the new MN vicinity may observe its lower rank and select the MN as their PP, even though the MN is no longer in a stable position. In this scenario, node 8 disconnects from node 6 and selects MN 2 as PP.

In Fig. 4, the same DODAG topology is deployed with nine nodes. Node 0 serves as the fixed sink with a rank of 0. Nodes 1 to 3 are fixed with rank 1, and Nodes 4 to 7 are fixed with rank 2. Node 8 is mobile. At T_0, MN 8 has a rank of 3 and is connected to node 6 as its PP ($PP = 6$). At time T_1, the MN starts to move upward in the DODAG hierarchy, that is, toward the sink to a new location, which it reaches at T_2. As it moves, the MN enters the communication range of nodes with lower ranks (closer to the sink) and begins to receive DIO messages from these new neighbors. To adapt to its new position, the MN must perform several hand-off processes, i.e., switching parent nodes to select the most suitable parent based on updated link metrics. However, due to the behavior of the standard trickle timer, which regulates the frequency of DIO message transmissions, topology convergence is delayed.

Consequently, MNs in an IWSN can experience several issues when using RPL without mobility support:

- *Outdated routing information:* frequent displacement causes nodes to retain outdated parent relationships, resulting in link failures and node disconnections.
- *Packet loss:* frequent transmission failures caused by disconnecting between MNs and their parent nodes increase the packet loss.
- *Energy consumption:* failed routes and packet loss induce constant retransmissions, which in turn cause increasing energy consumption and reduced network lifetime.

- *Latency:* when MNs are unable to select the optimal next hop due to topology inconsistencies introduced by hand-off mechanisms, packet forwarding becomes inefficient. This inefficiency, combined with repeated control message exchanges, contributes significantly to increased latency.
- *Loop creation:* previous child nodes can be chosen as new parents, which can lead to the creation of loops in the network.

Noting these issues, RPL must dynamically adapt to the movement of the nodes to ensure stable communication, energy efficiency, and optimal latency by integrating mobility support. This is particularly vital in IIoT applications such as automated warehouses and smart factories, where devices frequently move.

To address the previously mentioned issues more effectively, we propose RIMS-RPL, an efficient routing protocol designed for mobile IIoT environments. RIMS-RPL integrates new mechanisms into the native RPL to enhance connectivity for MNs and improve the parent selection procedure. It ensures compatibility with the standard RPL routing protocol while offering enhanced performance in dynamic industrial IoT scenarios.

5 RIMS-RPL description

The proposed RIMS-RPL is based on the foundation of ME-RPL [30]. Although ME-RPL is an older proposal, it was chosen for several reasons:

- *Well-established:* ME-RPL is a well-known approach, with more than 160 citations from 2012 to 2024. It is straightforward to implement, making it easier to study and compare with the new proposition.
- *Innovative introduction of mobility status:* ME-RPL was one of the first protocols to introduce mobility status in DIO messages, favoring the selection of a fixed node as the next hop over a MN, improving network reliability. This method has since been widely adopted and recommended in subsequent research.
- *Efficient metrics for network stability:* ME-RPL introduced efficient metrics to track the number of parent changes, helping to monitor network stability in mobile environments. This aids decision-making when updating routing information.

However, this approach has some limitations, as discussed previously (see Sect. ref sec:RelatedWorkonAddressingMobilityinRPL). To address these limitations, improvements have been made by incorporating more intelligent methods while still utilizing the same key metrics (the number of parent changes and mobility status). These improvements target identified weaknesses, such as reducing the high number of control messages, improving the management of outdated information (as explained in Sect. 4), and introducing the BP selection method, which was absent in ME-RPL (ME-RPL retains the basic OF).

In the following, we describe our proposed RIMS-RPL, which integrates these enhancements. This includes the introduction of a new cross-layer metric, an updated OF to improve reliability, and a RL-based algorithm to detect mobility.

5.1 A new cross-layer based metric

Cross-layer design aims to facilitate information exchange across all levels of the protocol stack, enabling the development of protocols or mechanisms that transcend the constraints of isolated OSI model layers. This approach improves communication and coordination between different protocol layers, thus optimizing network performance and functionality [33, 34].

In this subsection, we suggest exploiting both RPL and IEEE 802.15.4 in mobility prediction and the BP selection process. To this end, CSMA/CA is adopted for the MAC layer as the IEEE 802.15.4 standard. It is based on the unslotted, non-beacon-enabled mode [35]. Using CSMA/CA, after sending the packet, the sender waits for an ACK from the receiver to confirm the successful reception. If the sender does not receive an ACK within a specified timeout period, it assumes that the packet was not successfully received by the receiver or that the ACK was lost in transit. In this case, the sender initiates a retransmission of the same data packet. Once the predetermined threshold for failed retransmissions is reached, the packet is discarded.

In this work, we introduce a new approach that operates between the MAC and routing layers. Specifically, the CSMA-based MAC monitors the number of successfully transmitted packets and those that are unsuccessfully transmitted by each node to its destination. Subsequently, the routing layer employs these values to calculate the ERP using Equation (1) for the BP selection process (see Sect. 5.2) and Eq. (2) for mobility support (refers to Sect. 5.3).

$$ERP_n^{Nbr(n)}(\%) = \frac{Count(FTx_n^{Nbr(n)})}{Count(FTx_n^{Nbr(n)}) + Count(Tx_n^{Nbr(n)})} * 100 \tag{1}$$

where

$Nbr(n)$ is the set of neighbors of node n,
$ERP_n^{Nbr(n)}$ is the ERP of node n with all its neighbors,
$Count(FTx_n^{Nbr(n)})$ is the number of packets that have failed to reach the destination,

and $Count(Tx_n^{Nbr(n)})$ is the number of packets successfully transmitted (supposed by the MAC layer) by a node n to its neighbors $Nbr(n)$.

$$ERP_n^{BP(n)}(\%) = \frac{Count(FTx_n^{BP(n)})}{Count(FTx_n^{BP(n)}) + Count(Tx_n^{BP(n)})} * 100 \tag{2}$$

where

$BP(n)$ is the current BP of node n,
$ERP_n^{BP(n)}$ is the ERP of node n with its BP,
$Count(FTx_n^{BP(n)})$ is the number of packets transmitted by node n but failed to reach $BP(n)$ (discarded packet),

and $Count(Tx_n^{BP(n)})$ is the number of packets successfully transmitted by a node n to its $BP(n)$.

Note that the $ERP_n^{Nbr(n)}$ metric is crucial in the BP selection procedure because it helps the nodes to choose the most reliable parent among their neighbors. Specifically, an increasing $ERP_n^{Nbr(n)}$ for a node n indicates a high frequency of unsuccessful packet transmissions, highlighting the unreliability. This unreliability can lead to increased latency, reduced throughput, and higher energy consumption due to repeated transmission attempts, ultimately resulting in discarded packets.

On the other hand, the $ERP_n^{BP(n)}$ metric helps the nodes to predict potential disconnections between themselves and their parents. Precisely, an increasing value of $ERP_n^{BP(n)}$ for a node n indicates frequent unsuccessful packet transmissions to its designated parent and may indicate that the current node is distant from its parent. In such scenarios, the probability that the node will be disconnected from both its parent and the network increases. This metric adeptly detects node mobility, preventing prolonged disconnection of MNs and substantially reducing overall packet loss.

5.2 Best parent selection procedure

In RIMS-RPL, a new mobility-aware OF is designed using four metric types, which are combined to meet the requirements for QoS and select the best route. These metrics are the path's ERP, the mobility status, the ETX, and the RSSI. In the following, we will discuss each metric that has been used in the OF.

5.2.1 ERP of the path

We define the ERP of the path from node n to the root as the maximum ERP value among all nodes along this path. The root node always announces a minimum value of the ERP of the path. In particular, $MIN\_ERP = 0$, under the assumption that it is the most reliable node. Each node then calculates the ERP of the path that traverses through its selected BP, which is the path originating from the node itself. More formally, the ERP of the path originating from the node n is defined by Equation (3):

$$Max(ERP_{p(n)}^{Nbr(p(n))}, ERP_n^{Nbr(n)}) \tag{3}$$

Using the min-max method, the ERP of the path helps nodes avoid routes with a high number of failed transmissions, thereby enhancing network reliability and overall performance.

5.2.2 Mobility status

Mobile IoT applications encompass both MNs and static nodes. Recent mobility-aware adaptations of RPL have improved network reliability by distinguishing between mobile and static nodes. Static nodes can offer more stable connections to the sink. In this context, priority should be given to static nodes as potential PP candidates. This prioritization helps MNs successfully transmit their packets to their destinations while providing increased stability for static nodes. To facilitate this, a mobility status value is incorporated into the DIO message, where 0 indicates that the sensor is attached to a static object and 1 signifies that the sensor node is attached to a mobile object (as proposed in [26, 30]).

5.2.3 RSSI

The RSSI acts as a link stability indicator. The chance of communication failures is reduced by stronger signals. Hence, its use is essential to ensure consistent connectivity. Unlike other state-of-the-art approaches that rely on RSSI as the primary metric for mobility detection, our method combines RSSI with additional metrics to mitigate ambiguities caused by interference and environmental variability. In our application, this metric is typically measured at the physical layer, where the Chipcon CC2420 radio module returns a range of [-100, 0] for RSSI, where -100 dB represents the lowest level of background noise.

5.2.4 ETX

ETX is the expected transmission count (as defined in RFC6551 [21]). It estimates the average number of transmissions required by a node to successfully deliver a packet to its destination. A higher ETX value indicates poorer link quality. The inclusion of the ETX metric guarantees reliable packet delivery and energy efficiency by avoiding lossy links.

Algorithm 1 shows how these metrics are combined to calculate the routing path metric and the BP selection process.

Require: DIO from the CP m.
Ensure: the BP.
/* The following code is running on the child node n */
1: **if** Receive(DIO_m) from CP node m **then**
2: $m.ERP \leftarrow DIO_m.MC.ERP$
3: $m.MobilityStatus \leftarrow DIO_m.MC.MobilityStatus$
4: $m.Rank \leftarrow DIO_m.Rank$
5: $Calculate(m.RSSI)$
6: $Calculate(m.ETX)$
7: $m.PathMetric \leftarrow DIO_m.MC.PathMetric + \alpha \times (m.ETX) - \beta(m.ERP \times m.RSSI)$
8: **if** BP is $NULL$ **then**
9: $BP \leftarrow m$
10: **else**
11: **if** $BP.MobilityStatus = m.MobilityStatus$ **then**
12: $x \leftarrow BP.PathMetric$
13: $y \leftarrow PathMetric_{Threshold}$
14: **if** $m.PathMetric \leq (x - y)$ **then**
15: $BP \leftarrow m$
16: **end if**
17: **else**
18: **if** $(m.MobilityStatus = 0)$ **then**
19: $BP \leftarrow m$
20: **end if**
21: **end if**
22: **end if**
23: $n.PathMetric \leftarrow BP.PathMetric$
24: $n.Rank \leftarrow BP.Rank + \alpha \times (BP.ETX) - \beta(BP.ERP \times BP.RSSI)$
25: /* Generate and send a new DIO */
26: $DIO_n.MC.ERP \leftarrow Calculate(ERP)$ ▷ Using Equations (1) and (3)
27: $DIO_n.MC.MobilityState \leftarrow n.MobilityState$
28: $DIO_n.MC.PathMetric \leftarrow n.PathMetric$
29: $Send(DIO_n)$
30: **end if**

We assign the weights α and β (line 7) to the composite OF in order to balance multiple metrics when selecting the BP; more precisely, we aim to minimize ETX, maximize RSSI, and reduce path ERP. These weights were chosen empirically based on performance patterns observed across diverse simulation scenarios. The selected values reflect a trade-off that favors link quality under moderate mobility conditions while preserving energy efficiency. Although not exhaustively optimized, this configuration consistently yielded balanced and stable performance across the evaluated cases (refers to Sect. 6.3.5). When a node n receives multiple DIO messages from CPs in its neighborhood, it selects the neighbor with the lowest *PathMetric* value as the next hop. To prevent excessive

parent changes, a constant value, $PathMetric_{threshold}$, is used as a threshold (line 14). Note that the ERP value and the mobility status of each node are included in the DIO message to prevent the need for additional control messages to be generated.

5.3 RL-based approach for better connectivity in IIoT-based systems

Incorporating AI capabilities into communication networks remains an active area of research. RL, a widely embraced AI technique, has attracted considerable attention for its successful application in various network communication domains, such as channel access and route selection, to address several challenges (low complexity, low latency, low power consumption, high reliability, mobility, etc.) [36]. In RL, an agent or decision-maker, such as a sensor node, learns through interactions with its environment via the Markov decision process (MDP). The agent tries to make optimal decisions that maximize long-term rewards based on accumulated experiences [19, 20, 36]. Among the widely adopted RL techniques, Q-learning stands out. It has been commonly used to address challenges in WSNs (see, for example, [37, 38]), making a notable contribution to their advancement. In the Q-learning model, the agent n is configured with a set of states, denoted as $S = \{s_1, s_2, \ldots, s_n\}$, and a set of actions $A = \{a_1, a_2, \ldots, a_n\}$. When the agent is in a particular state $s \in S$, it performs a specific action $a \in A$ and receives a reward R_n. Based on the acquired reward, the agent moves to the next state $s' \in S$. The accumulated reward is stored in a *Q_table* as a *Q_value* for the corresponding state-action pair (s, a). This *Q_value* information is then utilized by the agent for future action selection in specific states using a greedy strategy to ensure a trade-off between the exploration of new actions and the exploitation of its learned knowledge.

In this work, the proposed Q-learning-based system is executed in a fully distributed manner across the nodes, allowing each node to independently learn and adapt based on its local observations and experiences. In this process, an IoT node n functions as an agent with three possible states that are s_0 = outdated information, s_1 = hand-off processes, and s_2 = stable state. Note that s_0 and s_1 refer to the states in which the scenarios illustrated in Figs. 3 and 4 (Sect. 4) occur, respectively. Additionally, the intelligent IoT node performs three actions that are a_0 = remove the current parent from the neighbor list, select a new parent, and reset its trickle timer; a_1 = broadcast a DIS message; and a_2 = no action.

After every new action, RIMS-RPL calculates the new ERP (using Equation (2)) and compares it with the old ERP (which was calculated before taking the action). If the new ERP is better than the old ERP, a positive reward R_n is received; otherwise, no reward is received. Based on the acquired reward, the agent updates the *Q_table* using *Q_value* for the associated state-action pair.

The proposed Q-learning-based system calculates the reward R_n for a particular node n using Equation (4).

$$R_n = \begin{cases} 1 \text{ if } New(ERP_n^{BP(n)}) \leq Old(ERP_n^{BP(n)}) \\ 0 \text{ Otherwise} \end{cases} \tag{4}$$

The reward R_n is used to measure the learning estimate $\delta Q(s,a)$ of a particular state-action pair, where $a \in A$ and $s \in S$. $\delta Q(s,a)$ is updated as given in Equation (5):

$$\delta Q(s,a) \leftarrow R(s,a) + \gamma\Big(\max_{a \in A} Q(s',a)\Big) - Q(s,a). \tag{5}$$

where $\max_{a \in A} Q(s',a)$ represents the estimated value for the next state-action pair.

$0 \leq \gamma \leq 1$ is the discount factor that sets the preference for immediately receiving rewards or deferring them. To be specific, when $\gamma = 0$, the agent prioritizes the current reward more prominently. In contrast, setting $\gamma = 1$ indicates that the agent prioritizes long-term rewards over immediate ones. After that, the calculated $\delta Q(s,a)$ is used to calculate the new *Q_value* using the Equation (6):

$$New\_Q(s,a) \leftarrow Q(s,a) + \alpha\Big(\delta Q(s,a)\Big) \tag{6}$$

where the learning rate $0 \leq \alpha \leq 1$ is used to adjust the speed of learning. When $\alpha = 0$, the agent sticks to what it already knows and does not learn anything new. On the other hand, when $\alpha = 1$, the agent focuses only on the latest information, ignoring what it knew before. The learning rate is important because it decides how fast the agent reaches the best possible value.

Table 1 summarizes the main components of the Q-learning algorithm used in our approach, highlighting key elements including the definitions of states and actions, the reward function, the exploration strategy, and important learning parameters.

Algorithm 2 describes the pseudocode of the proposed algorithm. In the proposed RIMS-RPL, the Q-learning-based approach is divided into four phases, as follows:

5.3.1 Activation phase

Q-learning is applied only if the current node or its BP is mobile (see line 2), which indicates an unstable network situation. The Q-learning algorithm is activated if the current node n encounters a degradation in the ERP value with its current parent (line 13). Once activated, the algorithm proceeds to the next phase.

Table 1 Summary of key components in the Q-learning-based approach

Q-Learning component	Description	Details
States	The set of all possible states the agent can be in.	s_0 = Outdated information, s_1 = Hand-off processes, s_2 = Stable state
Actions	Possible actions the agent can take.	a_0 = Remove the current parent from the neighbor list, select a new parent, and reset its trickle timer, a_1 = Broadcast a DIS message, a_2 = No action
Reward	Feedback signal to evaluate actions.	0 or 1, based on Equation (4)
Exploration strategy	Method used to balance exploration and exploitation	$\epsilon - greedy$ with condition-driven adaptive exploration strategy
Learning Parameters	Hyper parameters controlling the learning process	α and γ set empirically based on network conditions

Require: The BP.
Ensure: Optimal action.
Initialization: $BPCount\_c \leftarrow 0$, $BPCount\_u \leftarrow 0$, $action \leftarrow NULL$, $state \leftarrow s_2$, $Q\_table \leftarrow 0$, $\Delta Q \leftarrow 0$, $Old(ERP_n^{BP(n)}) \leftarrow 0$, $New(ERP_n^{BP(n)}) \leftarrow 0$.
/ The following code is running on the child node n */*
if Receive(DIO_m) from node m **then**
  if $(BP.MobilityStatus = 1) OR (n.MobilityStatus = 1)$ **then**
    if *BPchanged* **then**
      $BPCount\_c \leftarrow BPCount\_c + 1$
    else
      $BPCount\_u \leftarrow BPCount\_u + 1$
    end if
    $New(ERP_n^{BP(n)}) \leftarrow Calculate\_ERP(n, BP(n))$ ▷ Using Equation 2
    */********** Update phase **********/*
    if $action \neq NULL$ **then**
      Calculate_Reward(state, action) ▷ Using Equation 4
      Update_Qtable(state, action) ▷ Using Equation 5 and 6
    end if

    */********** Activation phase **********/*
    if $(New(ERP_n^{BP(n)}) > Old(ERP_n^{BP(n)}))$ **then**
      $Old(ERP_n^{BP(n)}) \leftarrow New(ERP_n^{BP(n)})$
      SELECT_ACTION($BPCount\_c, BPCount\_u$)
    end if
  end if
end if
procedure SELECT_ACTION($BPCount\_c, BPCount\_u$)
  */**** State prediction and action selection phase *****/*
  Pick a random number (k) in the interval [0,1]
  if $k \leq \epsilon$ **then**
  / Exploration */*
    if $(BPCount\_u > Threshold\_u)$ **then**
      $action \leftarrow a_0$
    else if $(BPCount\_c > Threshold\_c)$ **then**
      $action \leftarrow a_1$
    else
      $action \leftarrow a_2$
    end if
  else/* Exploitation */
    Select_optimal_action(state, action, Q_table) ▷ Using Equation 7.
  end if
  */**** Action execution and transition Phase ****/*
  if $action == a_0$ **then**
    Remove(BP)
    Select(newParent)
    Reset(DIO_Timer)
    $BPCount\_u \leftarrow 0$
    $state \leftarrow s_0$
  else if $action == a_1$ **then**
    Send(DIS)
    $BPCount\_c \leftarrow 0$
    $state \leftarrow s_1$
  else
    / No action to do */*
    $state \leftarrow s_2$
  end if
end procedure

5.3.2 State prediction and action selection phase

In this phase, it is necessary to check whether the current node is keeping outdated routing information (state s_0), undergoing hand-off processes (state s_1), or is in a stable situation (state s_2). To achieve this, Q-learning is employed to predict the current node's state by analyzing parent changes using pre-computed values (as proposed in [30]). Precisely, after receiving a new DIO message and updating its BP, the node checks whether the BP has changed. If a change is detected, it increments the *BPCount_c* counter, where *c* stands for changed. Otherwise, if the BP remains the same, it increments the *BPCount_u* counter, where *u* denotes unchanged (lines 3 to 7). Using these values, the proposed algorithm employs an $\epsilon - greedy$ with condition-driven adaptive exploration approach to make a trade-off between exploration and exploitation for action selection. In this approach, the agent explicitly explores the environment with a probability represented by epsilon (ϵ) and, for the rest of the time ($1 - \epsilon$), exploits existing knowledge. The algorithm randomly selects a number k between 0 and 1 to decide whether to exploit or explore.

During the exploitation phase, the agent relies on what it has learned *Q_value* and selects an action a_{best} using Equation (7) that has historically produced the highest accumulated positive reward in a given state.

$$a_{best} = arg_a max\Big(Q(s,a)\Big) \tag{7}$$

In the exploration phase, if the BP remains the same after a predefined number of *Threshold_u* messages (line 22), it may indicate outdated information or an unstable environment, and the node is likely moving away from its current parent in the opposite direction of the root (state s_0, which refers to the scenario illustrated in Fig. 3 (Sect. 4). In such a case, the node has a high probability of losing more packets in the future and should not wait for the next DIO period to update its parent set. Consequently, it selects a_0. On the other hand, if a node changes its BP several times successively (line 24), this may indicate that the current node is moving away from its current parent toward the root (state s_1, which refers to the scenario illustrated in Fig. 4 (Sect. 4). In this case, it selects a_1. Otherwise, the current node is likely to keep its parent and remain stable in the future (state s_2). In such cases, it selects a_2.

Figure 5 shows the state transition diagram of this phase. As described previously, the counters *BPCount_c* and *BPCount_u* track the number of times the BP changes (c) and remains unchanged (u), respectively. The parameters *Threshold_c* and *Threshold_u* are predefined thresholds corresponding to these counters and were empirically determined through multiple simulation runs.

In this model, when the agent (i.e., the IoT node) is in state s_0, meaning it has outdated information, it behaves as follows:

- If *BPCount_u* > *Threshold_u*, meaning the BP has remained unchanged for a long time, the agent stays in state s_0 to update its routing information by executing action a_0.

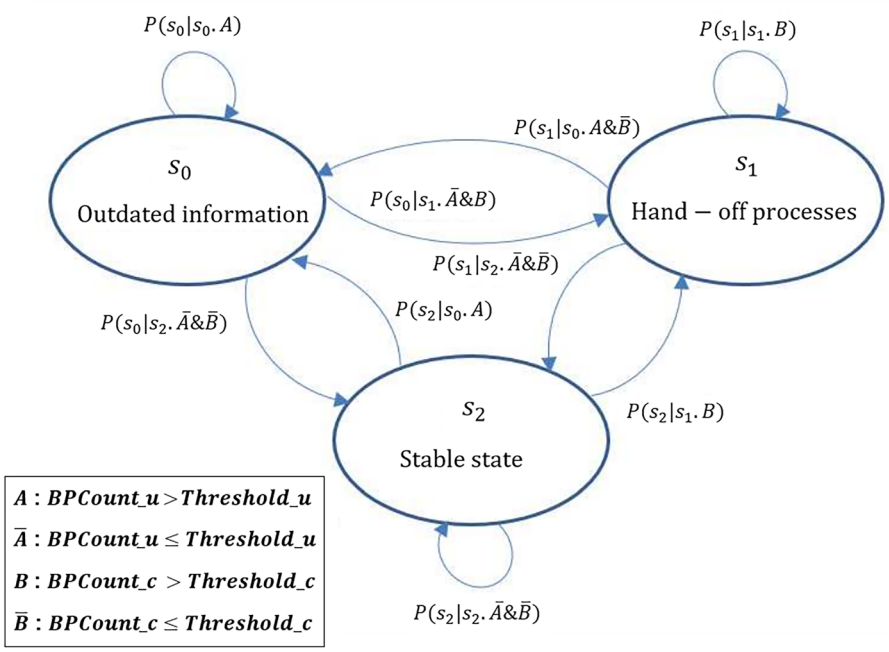

Fig. 5 State transition diagram of a Q-learning agent in RIMS-RPL, where an IoT node transits through states (s_0, s_1, s_2) using a condition-driven and adaptive exploration strategy guided by BP change counters (*BPCount_c*, *BPCount_u*)

- If *BPCount_c* > *Threshold_c* and *BPCount_u* ≤ *Threshold_u*, which means frequent parent changes have been detected, the agent moves to state s_1 and executes action a_1, to accelerate the process of achieving topology consistency.
- If both *BPCount_u* ≤ *Threshold_u* and *BPCount_c* ≤ *Threshold_c*, the agent switches to the stable state s_2, to minimize the control overhead by reducing the transmission of control messages.

Transition decisions are influenced by the exploration–exploitation trade-off of the agent. For example, P(s_0 | s_0,A), where A corresponds to the condition "*BPCount_u* > *Threshold_u*", represents the probability of remaining in s_0. If the random number $k < \epsilon$ (algorithm 2: line 21), the agent chooses to explore, staying at s_0; otherwise, if $k \geq \epsilon$, the agent exploits its learned policy and selects the optimal next state based on the Q_table.

Another example is the transition probability P(s_1 | s_0, $A\&\overline{B}$), where $A\&\overline{B}$ corresponds to the condition "(*BPCount_u* > *Threshold_u*) and (*BPCount_c* ≤ *Threshold_c*)". This denotes the probability that the agent transitions from state s_0 (indicating outdated routing information) to state s_1 (triggering hand-off processes). This transition is considered when the BP has remained unchanged for a prolonged period (*BPCount_u* > *Threshold_u*), while changes in parent selection have been relatively infrequent (*BPCount_c* ≤ *Threshold_c*). Under these

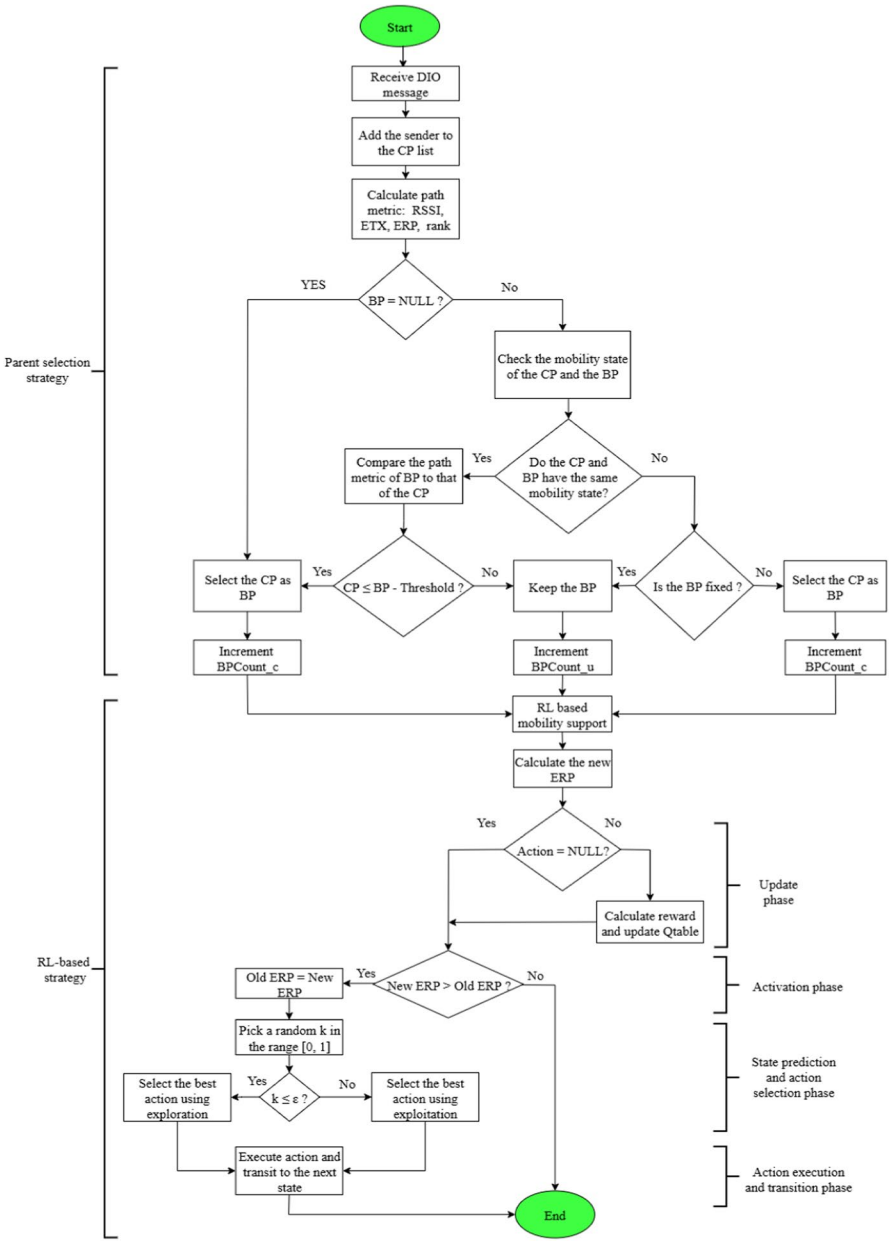

Fig. 6 Flowchart of the proposed RIMS-RPL approach

conditions, the agent interprets the routing behavior as insufficiently dynamic and transitions to the state s_1 to update the parent. Overall, the final decision depends on the agent's policy, which is guided by the Q-learning framework and shaped by the exploration-exploitation balance.

5.3.3 Action execution and transition phase

Based on the predicted state, the agent makes the decision to perform the action a_0, a_1, or a_2, and then transitions to a new state.

5.3.4 Update phase

After performing an action and transitioning to a new state, the agent calculates the reward using Eq. (4) and updates its Q_table using Eqs. (5) and (6). The Q_table is then used to select the optimal action during the exploitation phase, chosen by the greedy strategy.

Figure 6 illustrates the overall process of the proposed RIMS-RPL, including the parent selection procedure and the four phases of the RL-based strategy described above. It complements Algorithms 1 and 2 by providing a clear visual overview of the key decision steps and the learning process.

5.4 Complexity calculation

The algorithms proposed in this work result in lightweight complexity. This section discusses the computational complexity of each proposed algorithm, including space, time, and communication overhead.

5.4.1 Algorithm 1

1. *Time overhead:* the proposed algorithm assists nodes in selecting the shortest path to the destination by searching the neighbor table and identifying the best next hop node based on criteria such as the ERP of the path, mobility status, RSSI, and ETX. The computational complexity for selecting the next hop node is as follows:
   - If the current node is static: $O(n + m)$, where n is the number of 1-hop neighbors of the current node, and m is the number of MNs.
   - If the current node is mobile: $O(n)$, where n is the size of the network.
2. *Memory overhead:* two additional routing metrics were added to the DIO message, namely mobility status (1 byte) and the ERP of the path (2 bytes). It should be noted that while these additions introduce a slight increase in memory overhead, their impact is negligible when weighed against the substantial improvements achieved by our work compared to RPL, ME-RPL [30], and REFER [26] (refers to Sect. 6).

3. *Communication overhead:* this algorithm does not introduce additional messages compared to RPL to avoid unnecessary overhead.

5.4.2 Algorithm 2

1. *Time overhead:* unlike other state-of-the-art RL-based algorithms, our proposed RL-based algorithm does not require iterations to converge. In the activation phase, the Q-learning mechanism is activated only when the ERP value indicates unstable or suboptimal routing behavior. This reduces the frequency of Q_value updates, ensuring that learning is applied only when necessary. During the exploration phase, the agent selects the action associated with the highest Q_value from a fixed-size Q_table (3 states $\times$ 3 actions = 9 entries). Since only three actions are evaluated per state, this operation involves a simple search for the maximum value across three values. Therefore, the time complexity per decision step is: $O(s)(a) = O(3 \times 3)$, which results in O(1). This fixed and constant cost ensures fast decision-making, even in real-time settings. On the other hand, during exploitation, the agent selects the next state-action pair based on local observations; notably, the values of the unchanged BP counter (*BPCount_u*) and the changed BP counter (*BPCount_c*). These indicators are compared with fixed thresholds using conditional statements, without iteration or search. As such, the exploitation step also operates in constant time with a complexity of O(1). In general, the proposed mechanism is local and threshold-driven, making it computationally lightweight and suitable for resource-constrained IIoT devices. These properties directly support the claim of low time overhead and justify the protocol's scalability in dynamic environments.
2. *Memory overhead:* the structure of the standard RPL routing table remains unchanged. RIMS-RPL introduces an additional local Q_table at each node to support intelligent parent selection under mobility. This table is compact, with a configuration of three states and three actions, resulting in nine entries. Each entry stores a Q_value occupying two bytes, for a total memory footprint of only 18 bytes. The Q_table is maintained locally and is non-exchangeable, meaning that it is not shared or transmitted between nodes. This design ensures low communication overhead and preserves memory efficiency, making RIMS-RPL highly suitable for deployment on resource-constrained IoT devices. In addition, the utilization of a limited number of states and actions, along with a simple reward function, ensures that the memory overhead remains minimal. As a result, common challenges associated with RL, such as the curse of dimensionality and insufficient memory availability, are effectively mitigated.
3. *Communication overhead:* similarly to algorithm 1, this algorithm does not add any messages compared to RPL. All enhancements are integrated within the existing DIO message structure, ensuring protocol efficiency. As a result, the communication cost remains similar to that of RPL. The next section demonstrates, through simulation, that our improved protocol imposes less overhead in terms of latency, energy consumption, and control messages.

Table 2 Cooja simulation parameters

Parameters	Values
Operating system/Simulator	Contiki−2.7/Cooja
Radio medium model	Unit Disk Graph Medium
Simulation area	10,000 m^2
Mobility model	Random Waypoint
Mote type	Tmote Sky
Number of MNs	5, 10, 15, 20, 25, 30
DODAG root position	(50, 0)
Transmission/Interference range	Trans = 45 m, Inter = 70 m
Tx and Rx radio	TX = 80%, Rx = 70%
Speed limit	(0.5, 1, 2) m/s
Packet rate	(1, 3, 6, 10, 15, 20) packet/min
Transport layer	UDP
Radio duty cycle	ContikiMac
Physical layer	IEEE 802.15.4
Packet size	30 bytes
Learning rate	$\alpha = 0.4$
Discount factor	$\gamma = 0.3$
Exploration rate	$\epsilon = 0.01$

6 Performance evaluation

In this section, we analyze the performance of RIMS-RPL using a series of experiments under different simulation conditions. We demonstrate that the proposed protocol is highly suitable for IIoT-based systems, as it maintains high reliability, energy efficiency, and low latency while reducing network overhead.

6.1 System configuration

The proposed protocol was implemented and evaluated under the lightweight Contiki operating system and Cooja simulator, which were designed for resource-constrained IoT embedded devices. Note that the full implementation of the standard RPL is available. However, mobility is not supported by the original version of Cooja. For this reason, a mobility plugin [39] is added to Cooja to facilitate the simulation of mobile IoT networks. Furthermore, the BonnMotion tool [40] is used to generate the mobile movement traces of the nodes. In our simulation, we consider five fixed nodes, randomly placed within the sensing area to provide optimal connectivity. Alongside these fixed nodes, we include a varying number of MNs whose positions change according to the Random Waypoint model. A single static DODAG root is positioned in the bottom center of the sensing area to collect sensing data from all nodes.

Table 3 Typical operating conditions for tmote sky

Operating mode	Values
MCU on, Radio RX	21.8 mA
MCU on, Radio TX	19.5 mA
MCU on, Radio off	1800 μA
MCU idle, Radio off	54.5 μA

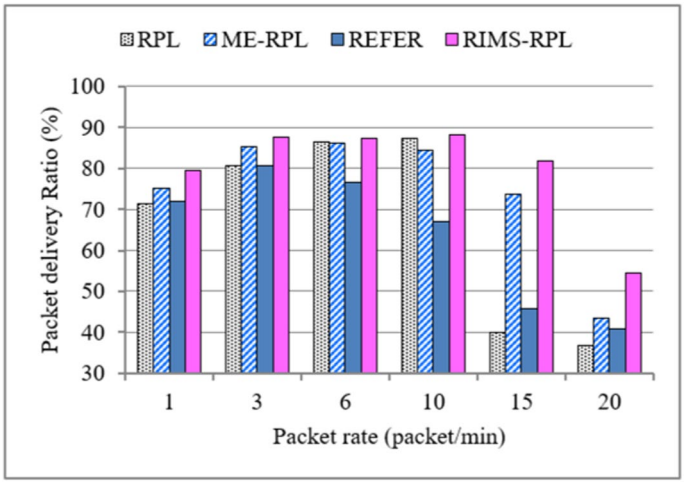

Fig. 7 The influence of data rate on PDR (with max speed = 1 m/s)

In our evaluation, we assume that all nodes (except the DODAG root) start with full battery energy, and each has the same initial energy level to ensure fairness in the evaluation. The DODAG root node is assumed to be mains-powered, and thus its energy is monitored during the simulation.

RIMS-RPL is evaluated and compared with the standard RPL, ME-RPL [30], and REFER [26] using different data rates, mobility speeds, and varied numbers of MNs. Detailed simulation parameters are listed in Table 2. It is important to note that the performance of the proposed protocol is sensitive to the choice of learning parameters (α, γ, and ϵ), and their optimal values can vary depending on the specific application context and network dynamics.

6.2 Performance metrics

In this evaluation, four main metrics are calculated to assess the performance of the proposed protocol as follows:

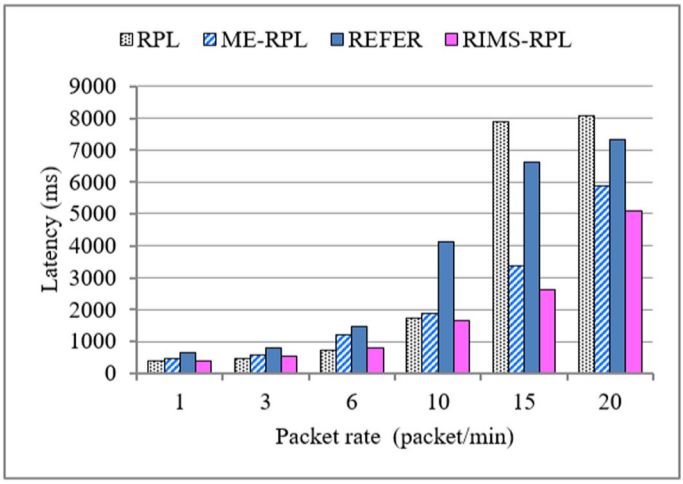

Fig. 8 The influence of data rate on latency (with max speed = 1 m/s)

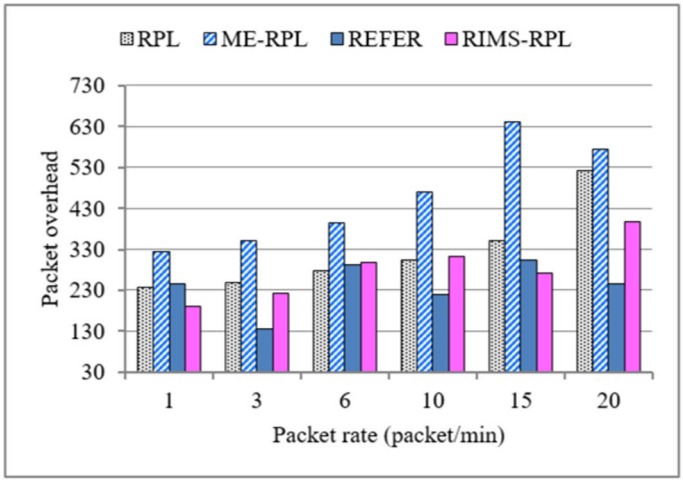

Fig. 9 The influence of data rate on packet overhead (with max speed = 1 m/s)

- *The PDR:* to evaluate the reliability of the network, we calculate the PDR, which is the ratio of the total number of packets successfully delivered to the sink node to the total number of packets sent by the nodes during the simulation time, as shown in Equation (8).

$$PDR(\%) = \frac{Total\ Packets\ Received\ by\ the\ Sink}{Total\ packet\ transmitted\ to\ the\ Sink} * 100 \tag{8}$$

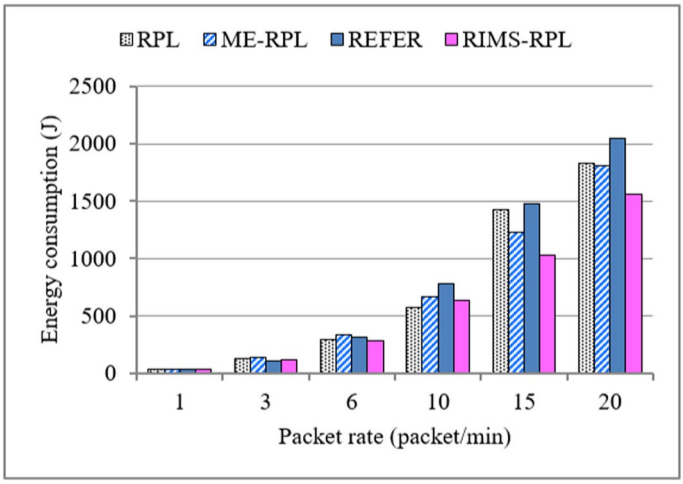

Fig. 10 The influence of data rate on energy consumption (with max speed = 1 m/s)

- *The average latency:* it is the average time taken for data packets to reach the sink node successfully, as depicted in Equation (9).

$$Average\ latency(ms) = \frac{\sum_{k=1}^{n}(Received\ time(k) - Sent\ time(k))}{Total\ packet\ received} \tag{9}$$

- *The packet overhead:* it represents the total number of ICMPv6 control messages (DIO, DIS, and DAO) exchanged between nodes for the DODAG construction during the simulation time. It is computed by Equation (10):

$$Packet\ overhead(packet) = \sum DIO + \sum DAO + \sum DIS \tag{10}$$

- *The power consumption:* to estimate the power consumption of the nodes, we use the Powertrace tool provided by Contiki. It is a linear power model that tracks the time the system spends in each power state (CPU in active state, CPU in sleep state, radio in listening state, and radio in transmission state). After that, we use the formula of Equation (11) (which is used in [37]) for the power consumption estimation:

$$Power\ consumption(mJ) = \frac{Energest\_Value * Current * Voltage}{RTIMER\_SECOND} \tag{11}$$

where,

Energest value is the difference in the number of ticks between two time intervals, measured in milliwatts.

Current is based on the Tmote Sky datasheet [41], as listed in Table 3.

The operational voltage of Tmote Sky is 3V, and RTIMER_SECOND is the number of ticks per second. In our simulation, RTIMER_SECOND has the value of 32,768 ticks/s.

6.3 Results and discussion

This section presents our simulation results and provides an in-depth discussion of their significance, followed by an extended statistical analysis to evaluate the relevance and stability of the observed performance differences between the proposed protocol and the baseline protocols (RPL, ME-RPL, and REFER).

6.3.1 Influence of data rate on performance metrics

Figures 7, 8, 9, and 10 illustrate the influence of data rate on the performance of the standard RPL, ME-RPL, REFER, and RIMS-RPL. In this simulation, we have fixed the number of MNs to ten, the maximum mobility speed to 1 m/s, and varied the data rate of the nodes (1, 3, 6, 10, 15, and 20 packets/min). High traffic impacts the performance of all protocols, as shown in Figs. 7, 8, 9, and 10, due to collisions and frequent packet retransmissions.

In Fig. 7, all protocols show high PDR at low to moderate rates (1–10 packets/min), but the PDR decreases as the traffic rate increases. However, RIMS-RPL shows an improvement of 17%, 13%, and 11% compared to RPL, REFER, and ME-RPL, respectively, at a data rate of 20 packets/min. RIMS-RPL prevents the degradation in ERP levels by refreshing the DODAG information and removing outdated parents from the parent list. Furthermore, the proposed reward function of the RL-based algorithm stimulates ERP improvement by providing positive rewards for choosing optimal actions.

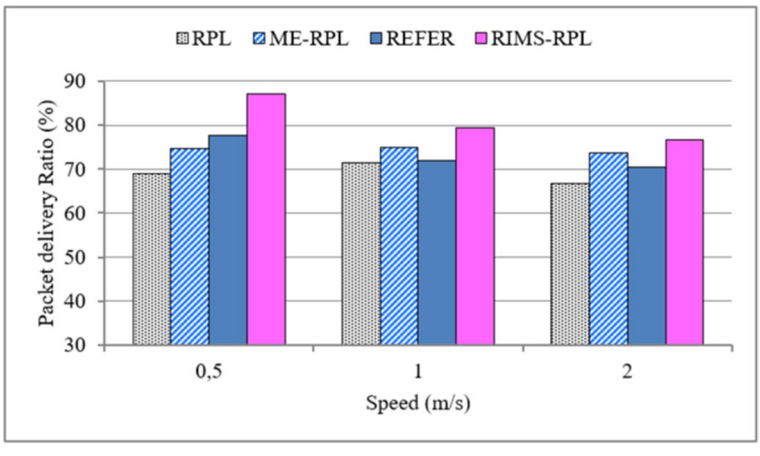

Fig. 11 The influence of speed on PDR (with packet rate = 1 packet/min)

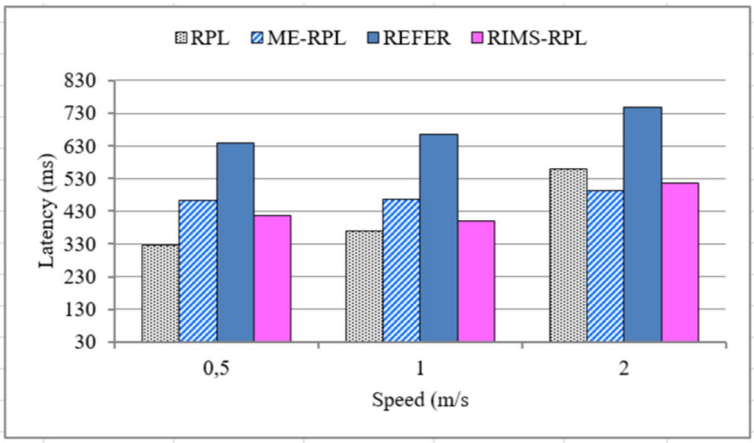

Fig. 12 The influence of speed on latency (with packet rate = 1 packet/min)

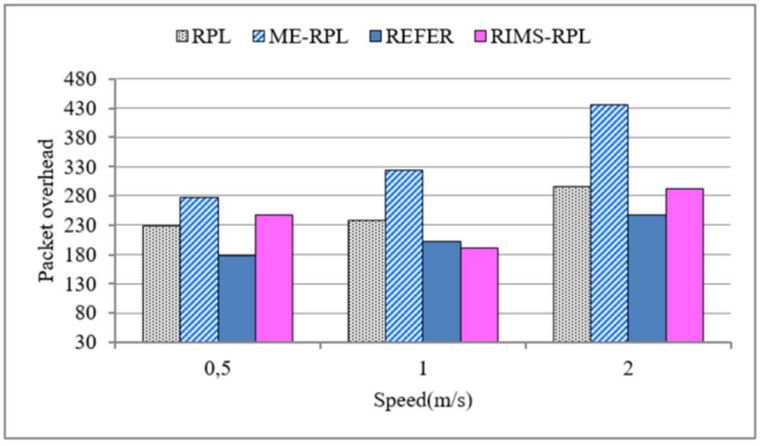

Fig. 13 The influence of speed on packet overhead (with packet rate = 1 packet/min)

Figure 8 displays the average latency of the four evaluated protocols. At low packet rates (1–6 packets/min), all protocols exhibit relatively acceptable latency. However, REFER already shows a noticeably longer delay as a result of its static-oriented routing strategy, which becomes inefficient under dynamic network conditions. ME-RPL consistently experiences higher latency than RIMS-RPL across different packet rates, primarily because of the overhead introduced by frequent control message exchanges. At higher traffic rates, both RPL and REFER suffer significant latency degradation compared to ME-RPL and RIMS-RPL, as they lack effective mechanisms for handling mobility and congestion. ME-RPL outperforms RPL in these scenarios thanks to more regular topology updates, but its control overhead still leads to increased delay.

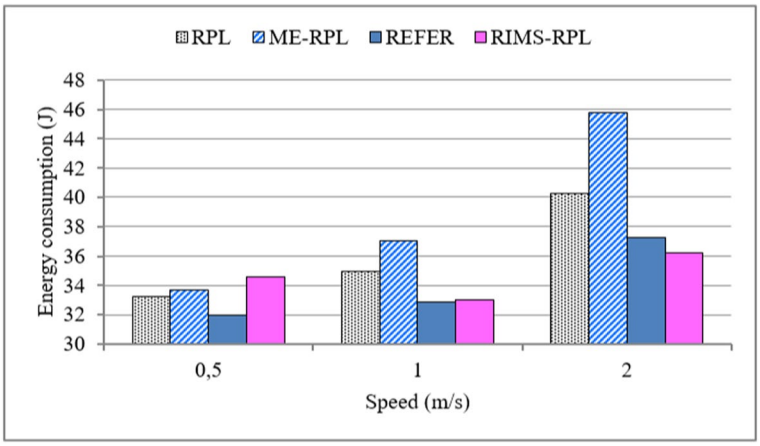

Fig. 14 The influence of speed on energy consumption (with packet rate = 1 packet/min)

In contrast, RIMS-RPL maintains the lowest end-to-end delay in all traffic conditions, particularly under high packet rates. This is achieved through dynamic parent selection based on current network conditions, including metrics such as RSSI, ERP, ETX, and node mobility status. By intelligently adapting to topology changes and avoiding congested or unstable routes, RIMS-RPL significantly reduces delivery delay and enhances overall network performance.

In Fig. 9, ME-RPL generates significantly more control messages than RPL, REFER, and the proposed RIMS-RPL, primarily because ME-RPL does not consider the phenomenon presented in Fig. 4, which is effectively addressed by RIMS-RPL. This results in persistent and excessive overhead, particularly visible at higher traffic rates (e.g., 640 packets at 15 packets/min), where ME-RPL exhibits the highest overhead across all protocols, which in turn directly contributes to increased energy consumption (Fig. 10), making it less suitable for energy-constrained IoT environments. In contrast, the proposed RIMS-RPL demonstrates consistently lower overhead than both RPL and ME-RPL, particularly under increased data traffic conditions, highlighting its efficiency in dynamic environments.

The performance of REFER is also notable, especially at lower to moderate traffic rates (e.g., at three packets/min, REFER generates only 135 control packets, the lowest among all protocols), indicating its suitability for sparse or moderately loaded networks. However, REFER shows fluctuating behavior at higher rates (e.g., 305 at 15 packets/min), suggesting potential instability in more congested or dynamic scenarios. Moreover, although REFER effectively reduces packet overhead, this does not necessarily translate to lower energy consumption. This is primarily due to its long end-to-end delays, which keep nodes active for extended periods, thereby increasing idle listening and transmission wait times. As a result, the protocol may still incur significant energy costs (as shown in Fig. 10) despite its lower volume of control messages, making it less efficient for time-sensitive or energy-critical applications.

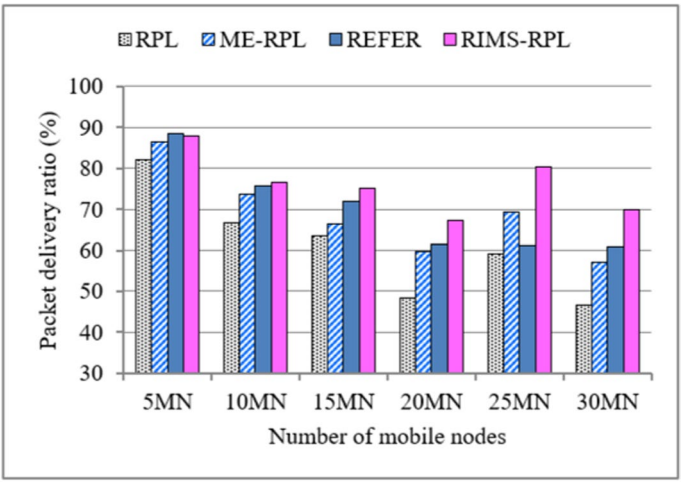

Fig. 15 The influence of the number of MNs on PDR (with max speed = 1 m/s and packet rate = 1 packet/min)

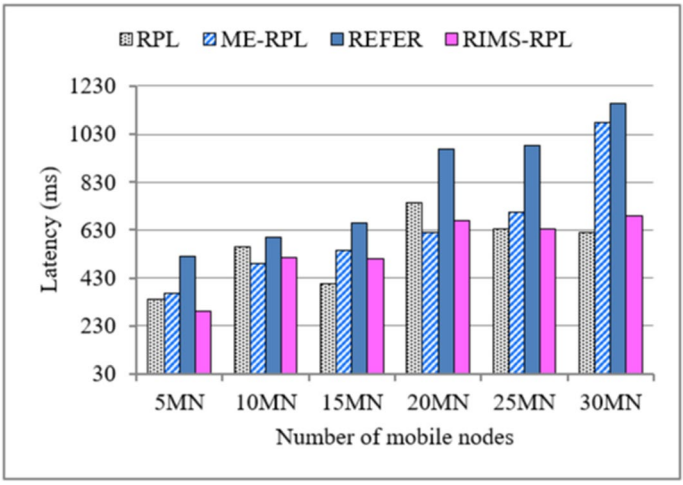

Fig. 16 The influence of the number of MNs on latency (with max speed = 1 m/s and packet rate = 1 packet/min)

The proposed RIMS-RPL outperforms all protocols in maintaining a balanced and adaptive control strategy in varying traffic loads. This is achieved through the RL-based algorithm that intelligently determines when to initiate control messaging. By evaluating the ERP level and predicting the states of MNs, RIMS-RPL selectively refreshes the DODAG information. This targeted control strategy significantly reduces unnecessary packet exchanges, optimizing the number of

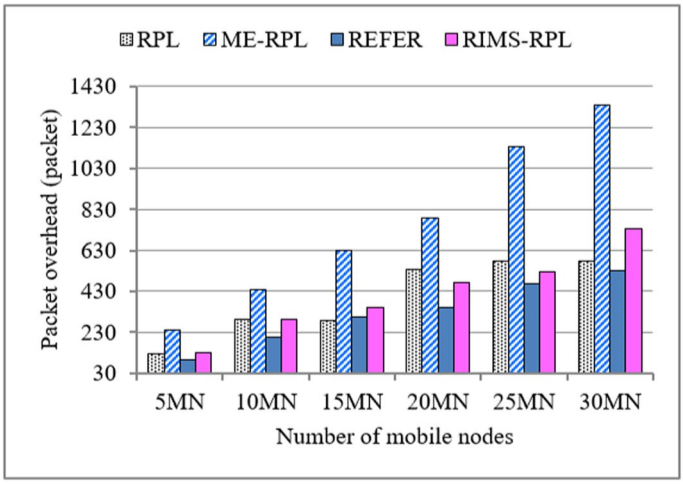

Fig. 17 The influence of the number of MNs on packet overhead (with max speed = 1 m/s and packet rate = 1 packet/min)

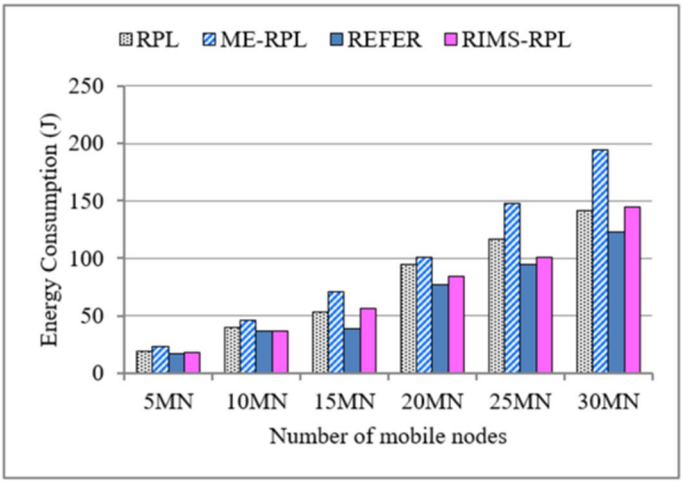

Fig. 18 The influence of the number of MNs on energy consumption (with max speed = 1 m/s and packet rate = 1 packet/min)

packet overhead exchanges in the network and thereby reducing energy consumption, as illustrated in Fig. 10.

6.3.2 Influence of speed on performance metrics

Figures 11, 12, 13, and 14 illustrate the influence of different MN speed limits on performance metrics. In this simulation, we have fixed the number of MNs to ten,

set the data rate to one packet/min (a low packet rate to minimize the influence of data traffic), and varied the speed limit of the MNs (0.5, 1, and 2 m/s).

In Fig. 11, the PDR of RIMS-RPL is significantly higher compared to RPL, ME-RPL, and REFER in all mobility scenarios evaluated. At a speed limit of 0.5 m/s, RIMS-RPL achieves an average improvement of 18%, 12%, and 9% over RPL, ME-RPL, and REFER, respectively. This improvement is primarily due to the fact that RIMS-RPL updates the DODAG topology quickly before link failures occur, which minimizes packet loss.

In contrast, RPL responds very slowly to topology changes, hindering its adaptability and performance in dynamic environments. Consequently, its PDR is limited to 66.66% when the MNs' speed is 2 m/s, whereas RIMS-RPL, REFER, and ME-RPL outperform it with improvements of 10%, 3%, and 7%, respectively, under the same simulation conditions. These results confirm that our learning-based strategy enables more resilient and adaptive routing, making it better suited for mobile and dynamic IoT environments.

In Fig. 13, ME-RPL incurs significantly higher packet overhead as the mobility speed of MNs increases, due to its frequent DODAG refreshes without effectively adapting to dynamic topology changes. This excessive control traffic leads to an increase in end-to-end latency in route establishment, as shown in Fig. 12, and results in higher energy consumption, as illustrated in Fig. 14.

By contrast, REFER consistently minimizes packet overhead, especially at lower mobility rates, which helps reduce energy consumption. However, this reduction comes at the cost of higher latency and less reliable packet delivery in more dynamic scenarios, as shown in Fig. 12. The proposed RIMS-RPL dynamically adjusts the frequency of control message transmissions based on the mobility state predicted by its RL-based algorithm. This predictive and selective update mechanism minimizes unnecessary communication, reducing both latency and packet overhead while maintaining high packet delivery reliability. Consequently, RIMS-RPL achieves superior energy efficiency, particularly under higher mobility conditions, as clearly depicted in Fig. 14. These results demonstrate that RIMS-RPL offers a more scalable and adaptive solution for mobile IoT scenarios compared to ME-RPL, RPL, and REFER.

6.3.3 Influence of the number of MNs on performance metrics

Figures 15, 16, 17, and 18 present simulation results under a fixed packet rate of 1 packet/min and a node speed limit of 1 m/s, while varying the number of MNs. As expected, increasing MN density negatively impacts the performance of all protocols due to more frequent topology changes and a reduced probability of maintaining stable routing paths.

In Fig. 15, RIMS-RPL consistently achieves a higher PDR in all network densities, outperforming RPL, ME-RPL, and REFER. This improvement is primarily due to its RL-based mechanism, which proactively updates the parent table of MNs by removing outdated parents and selecting new ones before disconnection

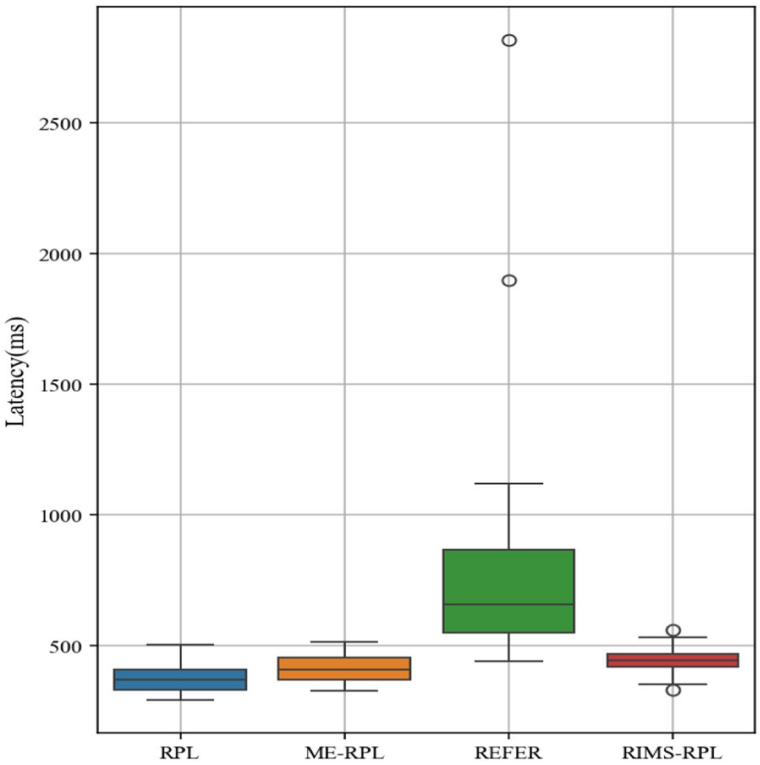

Fig. 19 Protocol performance comparison in terms of latency

occurs. Furthermore, the ERP level incorporated in the new OF improves the stability of the route, thereby reducing retransmissions and packet loss.

Figure 16 illustrates the increase in end-to-end delay as the number of MNs increases. Although latency increases for all protocols, ME-RPL and REFER are particularly affected in denser networks. Sending DIS messages in ME-RPL does not effectively address the issue of outdated parents (as highlighted in Fig. 4 of Sect. 4), which leads to an increased time to detect link failures and, consequently, to a longer delay in reaching the destination. A similar trend is observed for REFER, which exhibits the highest latency among the protocols at nearly all densities. This is mainly attributed to its passive parent selection strategy and infrequent updates, which cause MNs to rely on unstable routes and delay packet delivery.

As shown in Fig. 17, RIMS-RPL requires fewer control messages than RPL and ME-RPL to construct and maintain the network topology, contributing to its lower energy consumption depicted in Fig. 18. This efficiency comes from the intelligent and context-aware decisions made by the RL-based algorithm, which reduces unnecessary messaging and adapts to the behavior of the nodes more effectively.

In contrast, while ME-RPL achieves better PDR than the standard RPL, this improvement comes with a substantial increase in the overhead of the control

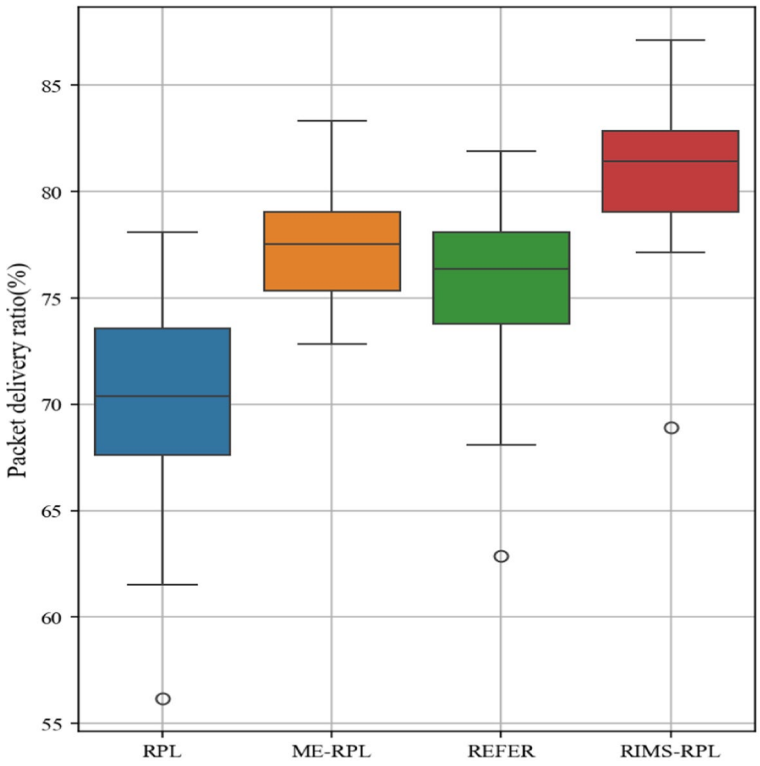

Fig. 20 Protocol performance comparison in terms of PDR

message, leading to greater energy consumption and increased latency. REFER, while minimizing overhead, does not adapt in dynamic conditions, resulting in high latency and inconsistent delivery performance.

In summary, RIMS-RPL demonstrates superior scalability and performance under increasing MNs density, achieving consistently higher PDR (Fig. 15), controlled latency (Fig. 16), reduced control overhead (Fig. 17), and improved energy efficiency (Fig. 18). These results confirm that RIMS-RPL provides a robust and energy-aware solution for mobile IoT environments.

6.3.4 Statistical analysis and performance variability

To complement the results presented previously, we performed a comprehensive statistical analysis to assess the variability, consistency, and significance of the performance differences between the proposed protocol and the baseline protocols (RPL, ME-RPL, and REFER). In particular, we conducted simulations for our proposed protocol and the three baseline protocols using the following setup:

Table 4 Summary of average performance metrics and statistical significance of differences compared to the proposed protocol

Metric	Latency	PDR	Packet overhead	Energy consumption
RPL	375.20 ± 50.26	70.30 ± 4.74	242.77 ± 25.58	35.62 ± 2.98
ME-RPL	412.71 ± 53.49	77.66 ± 2.81	303.17 ± 63.76	36.06 ± 3.87
REFER	790.35 ± 476.70	75.47 ± 4.26	180.40 ± 26.18	30.59 ± 1.78
RIMS-RPL	445.45 ± 51.93	81.03 ± 3.45	227.00 ± 27.26	32.24 ± 2.23
Statistically significant vs RIMS-RPL	REFER ($p = 0.0004$)	RPL ($p < 0.0001$), ME-RPL ($p = 0,0001$), REFER ($p < 0.0001$)	RPL ($p < 0.0001$), ME-RPL ($p < 0.0001$)	RPL ($p = 0,0127$), ME-RPL ($p < 0.0001$)

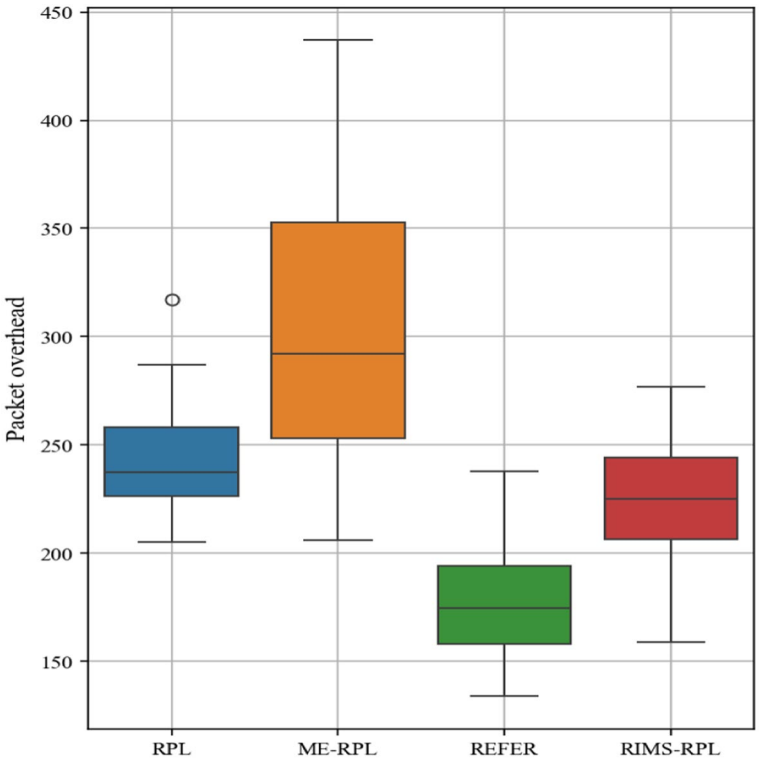

Fig. 21 Protocol performance comparison in terms of packet overhead

- We tested three different network topologies: grid, linear, and random topology.
- Each topology (scenario) involved five fixed nodes and 10 MNs.
- For each scenario, we performed 10 independent simulations, each with different random seeds, to account for variability.

For each protocol, we calculated the mean and standard deviation of the four performance metrics (latency, PDR, packet overhead, and energy consumption). Furthermore, we performed paired t-tests to assess the observed improvements and to determine whether the observed differences between our protocol and the baselines are statistically significant.

Table 4 summarizes the average results (mean $\pm$ standard deviation) for each metric, while the last column indicates whether the difference compared to our protocol is statistically significant ($p < 0.05$). Furthermore, Figs. 19, 20, 21 and 22 present the corresponding box plots, which visually highlight the results' variability and distribution of each metric across the 30 simulation runs. The following is a discussion of the results presented in the box plots.

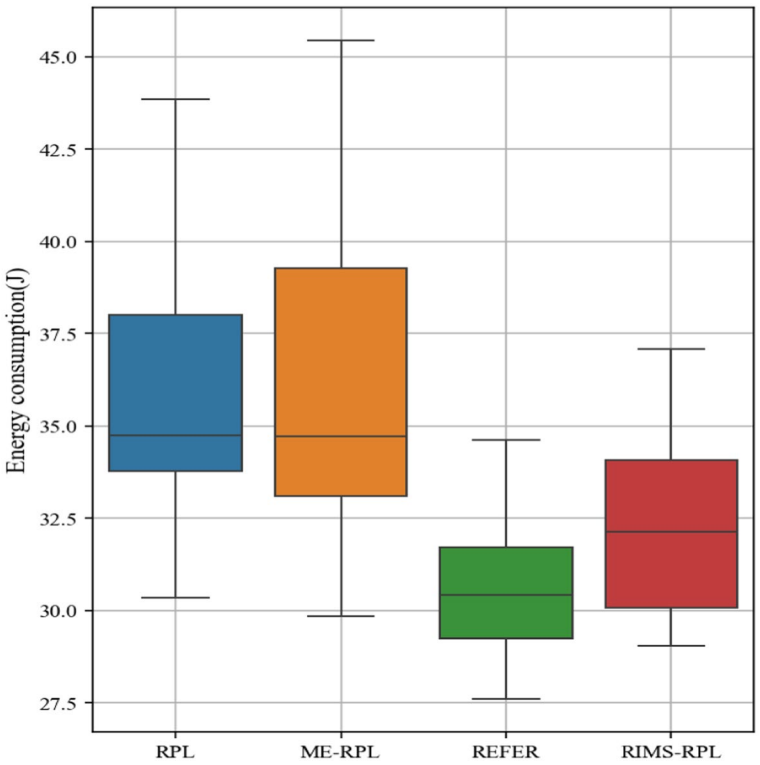

Fig. 22 Protocol performance comparison in terms of energy consumption

- Latency in RIMS-RPL remains statistically comparable to that of RPL and ME-RPL while being significantly lower than that of REFER, which suffers from high latency due to a suboptimal parent selection procedure. This procedure relies on a single prioritized metric rather than a combination of metrics and fails to consider the cumulative path cost during route computation.
- In Fig. 20, our proposed protocol achieves a significantly higher PDR than all baseline protocols ($p < 0.0001$), thanks to the use of the ERP metric, which monitors packet loss, and the RL-based strategy that effectively reduces node disconnections. ME-RPL demonstrates relatively good PDR performance due to its frequent control message exchanges that help maintain up-to-date topology information. The REFER protocol follows, benefiting from the suppression of unnecessary control messages, which helps maintain stability in static scenarios. Moreover, RPL, which lacks mobility support, shows the lowest PDR due to its limited adaptability to dynamic network conditions.
- In Fig. 21, the control overhead in RIMS-RPL is significantly reduced compared to RPL and ME-RPL ($p < 0.0001$). However, REFER slightly outperforms our protocol in this metric, as it only accepts DIO messages from other

static nodes. This selective acceptance minimizes transceiver activity related to control packets in stationary nodes, thereby reducing overhead. In contrast, our protocol prioritizes maintaining accurate and up-to-date topological information on static and MNs. While this leads to slightly higher overhead than REFER, it enhances adaptability and route stability in dynamic network conditions, offering a better balance between control efficiency and performance in mobile scenarios.

- In Fig. 22, RIMS-RPL achieves lower energy consumption compared to RPL and ME-RPL, primarily due to our strategy that minimizes frequent packet retransmissions and reduces unnecessary communication overhead. However, the REFER protocol consumes slightly less energy than RIMS-RPL, owing to its reduced communication overhead resulting from selective message processing. However, RIMS-RPL offers a more balanced trade-off between PDR, latency, and energy consumption, making it more suitable for dynamic and performance-critical scenarios. These findings confirm that the proposed protocol provides robust and consistent performance across varying simulation runs while significantly improving PDR and energy consumption.

6.3.5 Q-learning parameter sensitivity

To evaluate how RL parameters influence the performance of our protocol, we ran controlled experiments by varying the learning rate (α), discount factor (γ), and exploration rate (ϵ). Although visual graphs are omitted, we favor a concise yet informative textual analysis to highlight each parameter's realistic influence on learning behavior in practical IIoT environments.

- *Learning rate* (α): a higher learning rate (e.g., 0.9) allows the agent to learn from recent experiences quickly, facilitating adjustment to environmental changes at a faster rate. In highly dynamic networks, this would, however, result in unstable Q-value updates due to overreaction to short-term fluctuation. Lower α values (e.g., 0.1) result in slower but smoother convergence, offering higher stability in relatively static environments.
- *Discount factor* (γ): a high discount factor (e.g., 0.9) encourages long-term reward optimization, as wanted in stable topologies where future link quality is predictable. Smaller gamma values (e.g., 0.1), however, emphasize more immediate outcomes, making the system respond more quickly to topology changes at a high frequency, e.g., with MNs.
- *Exploration rate* (ϵ): although our exploration mechanism is condition driven rather than random, ϵ still determines how frequently the agent is allowed to explore alternative actions upon detecting instability. Higher values promote broader exploration in dynamic environments, finding better parent nodes under high mobility. Lower values favor exploitation of known good actions, accelerating convergence in stable environments.

These results validate the empirically chosen parameter values used in our implementation and offer practical advice for tailoring the protocol. Depending on the IIoT deployment context, ranging from mobile industrial robots to fixed sensor grids, various parameter settings can be used to optimize the trade-off between learning speed, stability, and responsiveness.

7 Real-world deployment considerations

While simulation results demonstrate that RIMS-RPL significantly enhances network stability and reduces control overhead and latency compared to RPL, ME-RPL, and REFER, real-world applicability remains a critical consideration. In this section, we present a practical use case scenario and examine the feasibility of implementing RIMS-RPL on resource-constrained IoT hardware platforms, such as Tmote Sky. We also address potential implementation challenges, including memory limitations, energy efficiency, processing constraints, and behavior of the MAC layer under real environmental conditions.

7.1 Real-world use case scenario

To demonstrate the practical applicability of the proposed protocol, we consider a real-world use case involving automated warehouse logistics, where mobile robots such as autonomous guided vehicles (AGVs) and robotic arms equipped with sensors are used to manage inventory, transport products, and coordinate real-time tracking. These robots move continuously within large indoor spaces, requiring real-time communication with central control units and other fixed infrastructure entities. The mobile nature of the system presents significant challenges for maintaining stable and efficient network connectivity. Efficient routing is essential to ensure reliable low-latency communication between these mobile entities and the network gateway. In this case, using the standard RPL for routing would result in a number of problems, including:

- Outdated routing tables caused by frequent topology changes due to robot mobility;
- Performance degradation due to increased packet loss;
- High-energy consumption due to frequent retransmissions of lost packets;
- Frequent disconnections will affect real-time data synchronization.

To address these issues, we have proposed RIMS-RPL, which will operate within automated warehouse logistics as follows:

- Each sensor node monitors packet transmission performance, including both successful and failed transmissions, to compute the cross-layer metric ERP.

- Calculates routing paths using the computed ERP, along with the received RSSI, ETX, and mobility status to capture link quality and node behavior.
- Selects the BP node using the proposed OF, which ensures reliable data delivery and stability under dynamic conditions.
- Tracks routing stability through two local counters. Precisely, when a sensor node or its BP is mobile, the node maintains two internal counters: the changed parent counter (*BPCount_c*) and the unchanged parent counter (*BPCount_u*). These counters reflect parent switch frequency and consistency.
- Triggers RL-based algorithm when a degradation in the ERP value is detected. In this step, the exploration-exploitation strategy leverages the computed counters *BPCount_c* and *BPCount_u* to update the Q_table. This allows the algorithm to select optimal actions, such as removing outdated parents, sending DIS messages, or resetting the trickle timer. These actions enable the node to proactively adapt to potential link failures and refine routing decisions before packet loss occurs, thereby reducing unnecessary retransmissions. As a result, the protocol ensures more efficient route selection and optimized energy consumption, ultimately extending the operational lifetime of battery-powered AGVs in the automated warehouse logistics system.

7.2 Integration architecture

A typical smart warehouse deployment involves 30 to 100 nodes, including fixed infrastructure sensors, central gateways, and mobile AGVs equipped with communication modules. Every AGV is also equipped with an embedded controller (e.g., Tmote Sky or Zolertia Z1) that talks to its onboard navigation and sensing systems. The nodes communicate with a border router or fog gateway, which typically serves as the sink node, located at the bottom center of the warehouse and forming a multi-hop mesh network topology.

RIMS-RPL is deployed as the routing layer on each node, replacing the default RPL implementation. It is embedded directly into both the network and MAC layers of the network stack, enabling deeper cross-layer optimization. Using the ERP metric and a Q-learning module, it makes local forwarding and medium access decisions without external coordination, making it ideal for decentralized systems like autonomous forklifts operating in confined indoor areas.

7.3 Compatibility with resource-constrained devices

RIMS-RPL is specifically designed to operate efficiently on resource-constrained IIoT devices. Its feasibility has been tested on the widely used Tmote Sky platform, which is equipped with a 16-bit MSP430 microcontroller, 10 KB of RAM, and 48 KB of Flash memory [41], making it an example of a highly resource-constrained device. The protocol is also expected to be compatible with similar hardware such as the Zolertia Z1, which features an MSP430F2617 microcontroller, 8 KB of RAM, and 92 KB of Flash memory [42], providing comparable processing capability with slightly more Flash storage.

- *Memory footprint:* The Q-learning module of the protocol uses a compact 3×3 Q_table (nine entries), stored as 16-bit integers, and requires only 18 bytes. While other internal variables such as counters and state indicators are also used, their memory usage remains minimal. The overall memory footprint of the learning component remains well within the limits of available RAM on platforms like Tmote Sky, ensuring compatibility with resource-constrained IIoT devices.
- *Processing load:* RIMS-RPL performs only basic arithmetic and logical operations without floating point or matrix operations, resulting in negligible CPU overhead on these systems.
- *Energy efficiency:* RIMS-RPL is designed to be energy-aware by minimizing control message exchanges during stable network states and avoiding unnecessary hand-off processes. Its Q-learning component is event-driven, meaning learning is triggered only when link degradation is detected via the ERP metric. This particular activation reduces unnecessary updates, parent changes, and data retransmissions, effectively decreasing energy consumption. While the added learning mechanism introduces minimal processing overhead, this is compensated by network control traffic reduction as a whole, as shown through our experimentation, and resulting in extended battery life for IIoT devices.

Overall, RIMS-RPL satisfies the computation, energy, and memory demands of resource-limited platforms like Tmote Sky and Zolertia Z1, demonstrating strong suitability for real deployment in real industrial environments.

7.4 MAC layer behavior in practice

In real-world deployments, MAC layer behavior may be affected by packet collisions and interference. These factors may result in delayed or missed ACKs, leading to differences between simulated and actual behavior. In order to simulate more realistic wireless conditions, we configured the simulation environment with the following parameters: a transmission success probability of 80%, and a reception success rate of 70%. While these settings introduce some degree of realism, they do not fully capture the complexity of physical layer and MAC layer dynamics present in real IIoT deployments.

This limitation is acknowledged, and future hardware experimentation is planned to incorporate real MAC layer behavior and environmental conditions in order to evaluate the robustness and reliability of RIMS-RPL under realistic constraints.

7.5 Expected benefits over the standard RPL

RIMS-RPL offers the following benefits compared to the standard RPL:

- *Improved reliability:* early detection of performance degradation by ERP value and RL-based strategies reduces packet loss in dynamic environments, leading to more stable and reliable communication.

- *Reduced latency:* predictive parent switching and updated routing information assist in faster route convergence and lower end-to-end delays, which is critical in real-time operations of autonomous forklifts and robotic arms.
- *Energy savings:* the protocol minimizes control message overhead and avoids unnecessary retransmissions, resulting in better energy efficiency and longer battery life for battery-operated mobile IIoT devices such as AGVs.
- *Mobility adaptation:* unlike the standard RPL, which degrades under high-speed topology changes, RIMS-RPL continuously learns and adapts to network dynamics, delivering more resilient communication for fast moving nodes in industrial environments.

7.6 Assumptions and limitations

It is important to note that our current simulation environment operates under several simplifying assumptions; these include:

- Modeling wireless communication with partial realism, incorporating probabilistic success rates but without explicitly accounting for real-world noise sources.
- Applying uniform energy consumption models, even though actual devices may consume energy differently.
- Considering synchronized timing, which may not reflect the asynchronous nature of real networks.

Although these assumptions are typical in early stage protocol evaluation, they will be addressed in future work by implementing and testing the proposed protocol on real IIoT platforms.

8 Conclusion and future work

This paper proposes an RIMS-RPL routing protocol, specially designed to support mobility in IIoT-based networks. The protocol introduces a new cross-layer metric that tracks failed transmission packets and employs a novel path selection strategy to ensure reliable routes using essential routing metrics, namely ETX, path's ERP, RSSI, and mobility state. In addition, a RL-based technique is proposed to predict mobility and make intelligent decisions to refresh the topology before disconnection. The computational complexity of the proposed protocol has been analyzed and proven to be lightweight and well-suited for constrained sensors. The proposed RIMS-RPL has been tested using the Cooja simulator under various simulation conditions and compared to the standard RPL, ME-RPL, and REFER. Extensive simulations demonstrate that our protocol outperforms RPL, ME-RPL, and REFER in terms of PDR, latency, packet overhead, and energy consumption. In conclusion, our protocol has proven to be efficient and adaptable, particularly for IIoT-based mobile networks and generally for energy-constrained devices. A valuable direction for future work would be to propose a new cross-layer metric that tracks transmission

delays for delay-sensitive IIoT applications. Furthermore, analyzing the behavior of RIMS-RPL in real-world IIoT deployments would further validate its effectiveness and adaptability.

Author Contributions Ismahene ALEM was contributed conceptualization, methodology, software, figures preparing, and writing manuscript. Samira YESSAD was involved in methodology, validation, and writing manuscript. Louiza BOUALLOUCHE-MEDJKOUNE was performed methodology, validation, and writing manuscript.

Funding Not applicable.

Data Availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Kalsoom T, Ramzan N, Ahmed S, Ur-Rehman M (2020) Advances in sensor technologies in the era of smart factory and industry 4.0. Sensors 20:6783. https://doi.org/10.3390/s20236783
2. Gliga L, Khemmar R, Chafouk H, Popescu D (2022) A survey of wireless communication technologies for an IoT-connected wind farm. Wireless Pers Commun 122(3):2253–2272. https://doi.org/10.1007/s11277-021-09013-w
3. Boyes H, Hallaq B, Cunningham J, Watson T (2018) The industrial internet of things (IIoT): An analysis framework. Comput Ind 101:1–12. https://doi.org/10.1016/j.compind.2018.04.015
4. Mechta D, Harous S, Alem I (2018) Improving wireless sensor networks durability through efficient sink motion strategy: TMSRP. Wireless Pers Commun 99(4):1661–1682. https://doi.org/10.1007/s11277-018-5804-y
5. Kaviani F, Soltanaghaei M (2022) CQARPL: Congestion and QoS-aware RPL for IoT applications under heavy traffic. J Supercomput 78(14):16136–16166. https://doi.org/10.1007/s11227-022-04488-2
6. Belkhiri-Brahmi L, Yessad S, Semchedine F (2024) Congestion control-based sink MOBility pattern for data gathering optimization in WSN. J Supercomput 80(3):3441–3479. https://doi.org/10.1007/s11227-023-05234-5
7. Sisinni E, Saifullah A, Han S, Jennehag U, Gidlund M (2018) Industrial Internet of Things: challenges opportunities, and directions. IEEE Trans Industr Inf 14:4724–4734. https://doi.org/10.1109/TII.2018.2852491
8. Dohler M, Watteyne T, Winter T, Barthel D (2009) Routing requirements for urban low-power and lossy networks. RFC 5548. https://doi.org/10.17487/RFC5548
9. Pister K, Thubert P, Dwars S, Phinney T (2009) Industrial routing requirements in low-power and lossy networks. RFC 5673. https://doi.org/10.17487/RFC5673
10. Brandt A, Buron J, Porcu G (2010) Home automation routing requirements in low-power and lossy networks. RFC 5826 https://doi.org/10.17487/RFC5826
11. Martocci J, De Mil P, Riou N, Vermeylen W (2010) Building automation routing requirements in low-power and lossy networks. RFC 5867. https://doi.org/10.17487/RFC5867
12. Azzaoui H, Boukhamla A, Perazzo P, Alazab M, Ravi V (2024) A lightweight cooperative intrusion detection system for RPL-based IoT. Wireless Pers Commun 134(4):2235–2258. https://doi.org/10.1007/s11277-023-10537-8
13. Kannan A, Selvi M, Santhosh Kumar S, Thangaramya K, Shalini S (2024) Machine learning based intelligent RPL attack detection system for IoT networks. In: Valadi J, Singh KP, Ojha M, Siarry P (eds) Advanced Machine Learning with Evolutionary and Metaheuristic Techniques. Springer, Berlin, pp 241–256. https://doi.org/10.1007/978-981-99-9718-3_10

14. Zahedy N, Barekatain B, Quintana A (2024) RI-RPL: a new high-quality RPL-based routing protocol using Q-learning algorithm. J Supercomput 80(6):7691–7749. https://doi.org/10.1007/s11227-023-05724-z
15. Darabkh KA, AlAdwan HH, Al-Akhras M, Jubair F, Rahamneh S (2024) A revolutionary RPL-based IoT routing protocol for monitoring building structural health in smart city domain utilizing equilibrium optimizer algorithm. Soft Comput 28(17):10099–10138. https://doi.org/10.1007/s00500-024-09677-0
16. Alam M, Ahmed N, Matam R, Mukherjee M, Barbhuiya FA (2023) SDN-based reconfigurable edge network architecture for industrial Internet of Things. IEEE Internet Things J 10(18):16494–16503. https://doi.org/10.1109/JIOT.2023.3268375
17. Gasmi R (2023) Deep reinforcement learning based robust communication for internet of vehicles. Autom Control Comput Sci 57(4):364–370. https://doi.org/10.3103/S014641162304003X
18. Li X, Liu H, Wang H (2024) Data transmission optimization in Edge computing using multi-objective reinforcement learning. J Supercomput 80(14):21179–21206. https://doi.org/10.1007/s11227-024-06213-7
19. Mammeri Z (2019) Reinforcement learning based routing in networks: review and classification of approaches. IEEE Access 7:55916–55950. https://doi.org/10.1109/ACCESS.2019.2913776
20. Sutton R, Barto A (1998) Reinforcement learning: an introduction. MIT Press (Bradford book) Cambridge
21. Winter T, Thubert P, Brandt A, Hui J, Kelsey R, Levis P, Pister K, Struik R, Vasseur J, Alexander R (2012) RPL: Ipv6 routing protocol for low-power and lossy networks. RFC 6550. https://doi.org/10.17487/RFC6550
22. Abdel Hakeem SA, Hady AA, Kim H (2019) RPL routing protocol performance in smart grid applications based wireless sensors: experimental and simulated analysis. Electronics 8(2):186. https://doi.org/10.3390/electronics8020186
23. Sanila A, Mahapatra B, Turuk AK (2020) Performance Evaluation of RPL protocol in a 6LoWPAN based Smart Home Environment. In: Proc. 2020 International Conference on Computer Science, Engineering and Applications (ICCSEA), Gunupur, India, IEEE, pp 1–6. https://doi.org/10.1109/ICCSEA49143.2020.9132942
24. Chen Y, Chanet JP, Hou KM (2012) RPL routing protocol a case study: precision agriculture. In: First China-France Workshop on Future Computing Technology (CF-WoFUCT), Harbin, China, pp 1-6. https://hal.science/hal-00681319/
25. Sanshi S, Jaidhar C (2019) Enhanced mobility aware routing protocol for Low power and Lossy Networks. Wireless Netw 25(4):1641–1655. https://doi.org/10.1007/s11276-018-1861-4
26. Lalani SR, Salehi AAM, Taghizadeh H, Safaei B, Monazzah AMH, Ejlali A (2020) REFER: A Reliable and Energy-Efficient RPL for Mobile IoT Applications. In: 2020 CSI/CPSSI International Symposium on Real-Time and Embedded Systems and Technologies (RTEST) IEEE, pp 1–8. https://doi.org/10.1109/RTEST49685.2020.9141192
27. Murali S, Jamalipour A (2018) Mobility-aware energy-efficient parent selection algorithm for low power and lossy networks. IEEE Internet Things J 6(2):2593–2601. https://doi.org/10.1109/JIOT.2018.2878677
28. Hoghooghi S, Javidan R (2020) Proposing a new method for improving RPL to support mobility in the internet of things. IET Netw 9(2):48–55. https://doi.org/10.1049/iet-net.2019.0152
29. Sanshi S, Jaidhar C (2019) Fuzzy optimised routing metric with mobility support for RPL. IET Commun 13(9):1253–1261. https://doi.org/10.1049/iet-com.2018.5562
30. El Korbi I, Brahim MB, Adjih C, Saidane LA (2012) Mobility Enhanced RPL for Wireless Sensor Networks. In: 2012 Third International Conference on The Network of the Future (NOF) IEEE. pp 1-8. https://doi.org/10.1109/NOF.2012.6378923
31. Seyfollahi A, Mainuddin M, Taami T, Ghaffari A (2024) RM-RPL: reliable mobility management framework for RPL-based IoT systems. Clust Comput 27(4):4449–4468
32. de Figueiredo MV, Kniess J (2019) Mobility aware RPL (MARPL): providing mobility support for RPL protocol. In: Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC), pp 211–223. https://doi.org/10.5753/sbrc.2019.7361
33. Yessad S, Bouallouche-Medjkoune L, Aïssani D (2015) A cross-layer routing protocol for balancing energy consumption in wireless sensor networks. Wireless Pers Commun 81(3):1303–1320
34. Srivastava V, Motani M (2005) Cross-layer design: a survey and the road ahead. IEEE Commun Mag 43(12):112–119

35. Alkama L, Bouallouche-Medjkoune L, Atmani M, Bachiri L (2022) Performance analysis of the unslotted IEEE 802.15.4k MAC protocols under saturated traffic and fading channel conditions. Computing 104(8):1891–1922
36. Alem I, Yessad S, Bouallouche-Medjkoune L (2023) Reinforcement learning based communication protocols for industrial WSNs: a critical mini-review. Colloque sur les Objets et Systèmes Connectés 2023 https://hal.science/hal-04219667
37. Nain Z, Musaddiq A, Qadri Y, Nauman A, Afzal M, Kim S (2021) RIATA: a reinforcement learning-based intelligent routing update scheme for future generation IoT networks. IEEE Access 9:81161–81172
38. Ahmed A, Abbas A, Rashid S, Jubair M, Hassan M, Abdulhadi A, Abdulsattar N, Habelalmateen M (2022) Congestion aware Q-learning (CAQL) in RPL protocol-WSN based IoT networks. In: 2022 5th International Conference on Engineering Technology and its Applications (IICETA), pp 429–435. IEEE
39. Osterlind F (2014) Mobility cooja plugin. Accessed: May. 20 2024.[Online]. Available: https://sourceforge.net/p/contikiprojects/code/HEAD/tree/sics.se/mobility/
40. Aschenbruck N, Ernst R, Gerhards-Padilla E, Schwamborn M (2010) BonnMotion: a mobility scenario generation and analysis tool. In: 3rd International ICST Conference on Simulation Tools and Techniques May 2010, pp 1–10
41. Corporation M (2006) Moteiv: tmote sky low power wireless sensor module. Prod, Data Sheet
42. Kharche S, Pawar S (2017) Effect of radio link and network layer parameters on performance of Zolertia Z1 motes based 6LoWPAN. In: 2017 8th International Conference on Computing, Communication and Networking Technologies (ICCCNT), pp 1–7. IEEE. https://doi.org/10.1109/ICCCNT.2017.8204038